



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ НА ТЕМУ:

*«Метод обеспечения согласованности состояний двух
асинхронных игроков с помощью конечных автоматов»*

Студент ИУ7-81Б
(Группа)

(Подпись, дата)

В. А. Лукьяненко
(И. О. Фамилия)

Руководитель ВКР

(Подпись, дата)

Д. А. Кузнецов
(И. О. Фамилия)

2026 г.

РЕФЕРАТ

Расчетно-пояснительная записка 89 с., 7 рис., 10 табл., 18 источн., 4 прил.

Расчетно-пояснительная записка содержит 89 страниц, 4 раздела, 7 рисунков, 10 таблиц, 19 использованных источников, 1 приложение.

В работе представлена разработка метода обеспечения согласованности состояний двух асинхронных игроков с помощью конечных автоматов. Проведен анализ подходов к синхронизации игровых состояний и передаче данных в многопользовательских приложениях, рассмотрены причины рассинхронизации при UDP-сетевом обмене. Разработан метод событийной синхронизации, включающий подтверждение обработки событий, повторную отправку неподтвержденных сообщений, контроль порядка и фильтрацию дубликатов. Выполнена программная реализация метода и проведено исследование его эффективности при задержках, потерях и дублировании UDP-пакетов. Проведено сравнение разработанного метода с базовой событийной UDP-синхронизацией без прикладной надежности.

КЛЮЧЕВЫЕ СЛОВА

согласованность состояний, многопользовательское приложение, распределенная асинхронная система, событийная синхронизация, конечный автомат, UDP, подтверждение доставки, повторная отправка, контроль порядка сообщений.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ВВЕДЕНИЕ	9
1 Аналитический раздел	11
1.1 Анализ предметной области синхронизации игровых состояний .	11
1.1.1 Общие положения сетевого взаимодействия	11
1.1.2 Игровое состояние и требования к его синхронизации . .	12
1.1.3 Модели передачи данных и источники рассинхронизации	13
1.1.4 Факторы, влияющие на требования к синхронизации . .	14
1.2 Формализация задачи обеспечения согласованности игровых данных	15
1.2.1 Математическое представление игрового состояния . . .	15
1.2.2 Модель клиент-серверного взаимодействия	16
1.2.3 Определение рассогласования состояния	17
1.2.4 Постановка задачи обеспечения согласованности	18
1.2.5 Требования к алгоритмам синхронизации	19
1.3 Обзор существующих подходов к синхронизации состояния . . .	20
1.3.1 Периодическая репликация состояния	20
1.3.2 Репликация только изменений	21
1.3.3 Событийная модель	21
1.3.4 Синхронизация по входным данным	22
1.3.5 Предсказание состояния на клиенте	22
1.3.6 Откат сетевого кода	23
1.4 Обзор методов передачи игровых данных	23
1.4.1 Надежная и ненадежная доставка данных	23
1.4.2 Классификация и приоритизация игровых пакетов	24
1.4.3 Механизмы обеспечения частичной надежности	25
1.4.4 Адаптивные алгоритмы передачи данных	25
1.5 Типовые игровые данные, подлежащие хранению, обновлению и передаче	26
1.5.1 Данные состояния объектов	26
1.5.2 Данные игровой логики	27

1.5.3	Данные взаимодействия и событий	27
1.5.4	Вспомогательные данные	28
1.5.5	Визуальные данные	28
1.5.6	Метаданные и технические данные	29
1.6	Основные причины возникновения рассинхронизации	30
1.6.1	Сетевые задержки	30
1.6.2	Потеря, дублирование и нарушение порядка доставки пакетов	30
1.6.3	Недетерминизм симуляции	31
1.6.4	Различия в производительности клиентских устройств	31
1.6.5	Неправильная или неполная сериализация данных	32
1.6.6	Несогласованный порядок выполнения событий	32
1.6.7	Ошибки в предсказании и последующей коррекции состояния	32
2	Конструкторский раздел	34
2.1	Требования к разрабатываемому методу	34
2.2	Функциональная модель метода	35
2.3	Формальная модель синхронизации	36
2.4	Состояния протокола синхронизации	39
2.5	Модель переходов протокола синхронизации	40
2.6	Структура передаваемых событий	42
2.7	Механизм подтверждения и повторной отправки	44
2.8	Контроль порядка сообщений	45
2.9	Алгоритм синхронизации состояний	45
2.10	Структура программного обеспечения	48
3	Технологический раздел	50
3.1	Выбор средств программной реализации	50
3.2	Структура программного обеспечения	50
3.3	Реализация сетевого обмена	52
3.4	Реализация структуры сетевого сообщения	57
3.5	Реализация менеджера синхронизации событий	60
3.6	Реализация подтверждений и повторной отправки	65
3.7	Реализация объектов с конечными автоматами	67

3.8	Реализация пользовательского взаимодействия	70
3.9	Средства имитации сетевых ошибок	70
3.10	Тестирование разработанного программного обеспечения	73
4	Исследовательский раздел	76
4.1	Цель исследования	76
4.2	Сравниваемые режимы синхронизации	76
4.3	Критерии оценки эффективности	77
4.4	Порядок проведения исследования	79
4.5	Сценарии исследования	79
4.6	Результаты исследования разработанного метода	80
4.7	Результаты исследования базового UDP-подхода	81
4.8	Сравнение результатов	82
4.9	Выводы по результатам исследования	83
	ЗАКЛЮЧЕНИЕ	85
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	88
	ПРИЛОЖЕНИЕ А	89

ВВЕДЕНИЕ

Многопользовательские игровые приложения относятся к классу распределенных интерактивных систем, в которых несколько участников взаимодействуют с общим виртуальным состоянием через сеть. Корректность работы таких систем зависит не только от скорости передачи данных, но и от согласованности состояний, хранящихся и обрабатываемых на разных сторонах взаимодействия.

В условиях асинхронного сетевого обмена состояние игроков может рассогласовываться из-за задержек передачи сообщений, потери пакетов, дублирования данных, нарушения порядка доставки и различий во времени обработки событий. Для приложений, использующих UDP, эта проблема становится особенно существенной, так как данный протокол не предоставляет встроенных гарантий доставки и упорядочивания сообщений.

Одним из способов решения данной задачи является представление взаимодействия игроков в виде событийной модели с формально заданными состояниями и переходами. Использование конечных автоматов позволяет ограничить множество допустимых состояний системы, определить правила обработки событий и обеспечить одинаковый порядок изменения состояния у участников взаимодействия.

Целью выпускной квалификационной работы является разработка метода обеспечения согласованности состояний двух асинхронных игроков с помощью конечных автоматов.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- провести анализ предметной области синхронизации игровых состояний и передачи данных в многопользовательских играх;
- рассмотреть существующие подходы к синхронизации состояний и способы передачи игровых данных;
- описать типовые игровые данные, подлежащие хранению и передаче;
- определить основные причины рассинхронизации игровых состояний;
- разработать метод синхронизации состояний двух игроков с использованием конечных автоматов;

- определить формальную модель состояний системы и переходов между ними;
- разработать структуру конечного автомата, описывающего взаимодействие игроков;
- представить алгоритм синхронизации состояний в виде схем алгоритмов или диаграмм состояний;
- обосновать выбор технологий и средств разработки программной реализации метода;
- разработать программное обеспечение, реализующее предложенный метод синхронизации;
- разработать программное обеспечение, реализующее взаимодействие игроков;
- провести тестирование разработанного программного обеспечения;
- определить критерии оценки эффективности разработанного метода;
- провести исследование эффективности разработанного метода;
- оценить скорость синхронизации состояний и устойчивость алгоритма к различным сценариям взаимодействия игроков;
- сравнить результаты работы разработанного метода с существующими подходами синхронизации состояний.

1 Аналитический раздел

1.1 Анализ предметной области синхронизации игровых состояний

1.1.1 Общие положения сетевого взаимодействия

Многопользовательские игровые приложения представляют собой распределенные системы, в которых несколько клиентов взаимодействуют с общим игровым миром через сеть [1; 2]. Для обеспечения корректности игрового процесса требуется поддерживать согласованность данных, описывающих состояние мира, действия игроков и результаты игровых событий. Сетевая передача данных сопровождается задержками, потерями пакетов, вариативностью задержки и ограничениями пропускной способности, что является одним из основных рисков возникновения рассинхронизации между участниками взаимодействия.

В отличие от одиночных игровых приложений, где состояние системы хранится и изменяется в пределах одного процесса, многопользовательское приложение должно поддерживать несколько локальных представлений одного и того же игрового мира. Каждое такое представление обновляется на основании входных действий игрока, локальной симуляции и сетевых сообщений, полученных от других участников сетевого обмена. Если порядок применения этих воздействий отличается, то даже при одинаковых исходных данных локальные состояния могут разойтись.

Наиболее распространенной архитектурой является модель с выделенным сервером, который хранит авторитетное состояние игрового мира [3]. Клиенты направляют серверу информацию о своих действиях и получают обратно обновления состояния. Такая модель упрощает контроль корректности, так как сервер может проверять допустимость действий, разрешать конфликты и рассылать согласованное состояние всем участникам. В то же время серверная архитектура не устраняет сетевые задержки и не исключает временное расхождение между состоянием, отображаемым на клиенте, и состоянием, принятым сервером.

1.1.2 Игровое состояние и требования к его синхронизации

Игровое состояние представляет собой совокупность параметров, описывающих положение объектов в игровом мире, их атрибуты и поведение. К таким параметрам относятся координаты и ориентация объектов, скорость движения, состояние анимации, логические флаги, параметры здоровья, ресурсы, активные эффекты и данные о текущей фазе игрового сценария. Часть параметров изменяется непрерывно или с высокой частотой, а часть изменяется дискретно в результате игровых событий.

Для синхронизации состояний существуют два свойства:

- согласованность, при которой клиентское представление состояния мира не должно существенно отличаться от авторитетного состояния;
- отзывчивость, при которой реакция на действия игрока должна быть визуально своевременной, несмотря на сетевые задержки.

Совместное обеспечение этих свойств представляет собой одну из основных проблем сетевых игр. Повышение отзывчивости часто достигается за счет локальной обработки ввода и временных предсказаний на клиенте. Однако такие предсказания могут не совпасть с результатом, полученным на авторитетной стороне, что приводит к необходимости корректировки состояния. В свою очередь, строгая ориентация только на подтвержденные сетевые данные уменьшает вероятность ошибки, но увеличивает задержку реакции приложения на действия пользователя.

Для игровых сценариев с непосредственным взаимодействием двух игроков особенно важны логически значимые события. В отличие от вспомогательных визуальных данных, такие события изменяют состояние системы: запускают действие, фиксируют столкновение, изменяют фазу взаимодействия, завершают ход или переводят объект в новое состояние. Потеря или повторная обработка такого события приводит не только к визуальной ошибке, но и к нарушению логики игрового процесса.

1.1.3 Модели передачи данных и источники рассинхронизации

Передача данных в многопользовательских играх может осуществляться по нескольким моделям:

- снимки состояния (snapshots) — периодическая отправка полного или частичного состояния объектов;
- событийная репликация (event-based sync) — передача только событий, влияющих на состояние мира;
- синхронизация по входным данным (input-sync) — передача входных команд игроков при локальном выполнении симуляции по детерминированным правилам.

Каждая модель имеет свои ограничения. Передача снимков состояния упрощает восстановление клиентского представления, но требует значительного объема трафика при большом количестве объектов. Событийная репликация экономит сетевые ресурсы, но зависит от корректной и одинаковой обработки событий на всех сторонах взаимодействия. Синхронизация по входным данным обеспечивает низкое сетевое потребление, однако предъявляет высокие требования к детерминизму симуляции и одинаковости вычислений на всех клиентах.

Основные источники рассинхронизации включают:

- сетевые задержки и их вариативность;
- потерю, дублирование и нарушение порядка доставки пакетов;
- недетерминированность клиентской симуляции;
- различия в частоте обновления и производительности устройств;
- асинхронность обработки событий.

Для рассматриваемой работы наибольшее значение имеет рассинхронизация, возникающая из-за неодинакового порядка обработки событий. Если одно событие переводит систему из состояния q_i в состояние q_j , то применение

другого события до завершения этого перехода может оказаться недопустимым. Следовательно, синхронизация должна включать не только передачу сообщения, но и проверку того, что событие применимо к текущему состоянию системы.

1.1.4 Факторы, влияющие на требования к синхронизации

Требования к точности и частоте синхронизации существенно отличаются в зависимости от жанра игры:

- FPS и экшен-игры требуют высокой частоты обновления и минимальной задержки, поскольку ошибки в позиции или времени приводят к критичным изменениям игрового процесса;
- MOBA и RPG допускают более низкую частоту синхронизации, поскольку взаимодействия менее чувствительны к малым задержкам;
- RTS часто используют детерминированную модель мира и обмен только входными данными для повышения масштабируемости.

Помимо жанра, важными факторами являются количество игроков и объектов в сцене, нагрузка на сеть, ограничения пропускной способности, необходимость поддержки слабых клиентских устройств и плотность взаимодействия игроков друг с другом. Чем выше плотность взаимодействия, тем больше вероятность конфликтов между событиями и тем строже должны быть правила их обработки.

В случае двух игроков требования к пропускной способности ниже, чем в массовых многопользовательских системах, однако требования к корректности обработки событий остаются высокими.

1.2 Формализация задачи обеспечения согласованности игровых данных

1.2.1 Математическое представление игрового состояния

Игровой мир многопользовательского приложения можно представить в виде динамической системы, состояние которой изменяется во времени под воздействием действий игроков и игровой логики. Состояние мира в момент времени t обозначим как $S(t)$:

$$S(t) = \{s_1(t), s_2(t), \dots, s_n(t)\},$$

где $s_i(t)$ — состояние i -й игровой сущности.

Каждое состояние можно формально представить как вектор параметров:

$$s_i(t) = (p_i(t), v_i(t), a_i(t), \lambda_i(t)),$$

где:

- $p_i(t)$ — позиция сущности;
- $v_i(t)$ — скорость;
- $a_i(t)$ — параметры активности или выполняемого действия;
- $\lambda_i(t)$ — дискретные состояния и флаги.

Все состояние мира представляет собой многомерное пространство, которое необходимо корректно поддерживать на взаимодействующих сторонах. Для двух игроков состояние можно представить как объединение локальных состояний игроков и состояния общих объектов:

$$S(t) = (S_1(t), S_2(t), S_c(t)),$$

где:

- $S_1(t)$ — состояние первого игрока;
- $S_2(t)$ — состояние второго игрока;

— $S_c(t)$ — состояние общих сущностей и параметров взаимодействия.

В событийной модели изменение состояния вызывается событием $e \in E$, где E — множество допустимых игровых событий. Тогда переход состояния можно описать функцией:

$$S(t + \Delta t) = F(S(t), e),$$

где F — функция применения события к текущему состоянию.

1.2.2 Модель клиент-серверного взаимодействия

В большинстве сетевых игр сервер является источником авторитетного состояния. Клиентская сторона может оперировать временными предсказаниями, но окончательные значения состояния принадлежат серверу. На стороне клиента поддерживается приближенное состояние $\hat{S}(t)$, которое может отличаться от серверного $S(t)$.

Клиент получает обновления состояния в дискретные моменты времени t_k :

$$S(t_k) \rightarrow \text{клиент},$$

а отправляет данные о действиях игрока в форме входных команд:

$$u(t_k) = \text{input}(t_k), \quad u(t) \in U,$$

где U — пространство возможных действий.

Сервер выполняет симуляцию:

$$S(t + \Delta t) = F(S(t), u_1(t), u_2(t), \dots, u_m(t)),$$

где F — функция игровой логики, m — количество игроков.

Для событийной синхронизации входная команда может рассматриваться как причина формирования события. В этом случае сетевое взаимодействие описывается передачей сообщения m_k , содержащего событие e_k , его идентификатор, номер последовательности и служебные параметры доставки. Если сообщение потеряно или обработано не в том порядке, то соответствующее событие может быть применено некорректно или не применено вовсе.

1.2.3 Определение рассогласования состояния

Пусть клиентское состояние $\hat{S}(t)$ является приближением авторитетного состояния $S(t)$. Тогда показатель рассогласования можно определить следующим образом:

$$D(S(t), \hat{S}(t)) = \sum_{i=1}^n w_i \cdot d(s_i(t), \hat{s}_i(t)),$$

где $d()$ — расстояние между состояниями одной сущности, w_i — вес значимости сущности.

Функция $d()$ может включать позиционную ошибку, ошибку скорости и различие дискретных параметров:

$$d(s_i, \hat{s}_i) = \alpha \cdot \|p_i - \hat{p}_i\| + \beta \cdot \|v_i - \hat{v}_i\| + \gamma \cdot \delta(\lambda_i, \hat{\lambda}_i),$$

где:

- $\|p_i - \hat{p}_i\|$ — расстояние между фактической позицией сущности и ее клиентским приближением;
- $\|v_i - \hat{v}_i\|$ — разница скоростей сущности;
- $\delta(\lambda_i, \hat{\lambda}_i)$ — бинарная функция различия дискретных параметров;
- α, β, γ — веса значимости соответствующих ошибок.

Бинарная функция различия дискретных параметров задается следующим образом:

$$\delta(\lambda_i, \hat{\lambda}_i) = \begin{cases} 0, & \text{если } \lambda_i = \hat{\lambda}_i, \\ 1, & \text{если } \lambda_i \neq \hat{\lambda}_i. \end{cases}$$

Допустимый порог рассогласования определим как:

$$D(S(t), \hat{S}(t)) \leq D_{\max}.$$

Если ошибка превышает порог, требуется корректировка состояния или повторная обработка событий, приведших к расхождению.

Для дискретной событийной логики существенна не только численная ошибка параметров, но и совпадение логических состояний. Если два игрока имеют одинаковые координаты объектов, но находятся в разных состояниях конечного автомата взаимодействия, например один клиент считает действие завершённым, а другой — выполняющимся, система остается несогласованной. Поэтому при разработке метода необходимо учитывать как параметрическое, так и логическое рассогласование.

1.2.4 Постановка задачи обеспечения согласованности

Целью системы синхронизации является минимизация рассогласования между локальными представлениями состояния при ограничениях на сетевые ресурсы и задержки:

$$\min_{\Pi} D \left(S(t), \hat{S}_{\Pi}(t) \right),$$

где Π — выбранный алгоритм синхронизации.

При этом должны выполняться следующие ограничения:

$$B(\Pi) \leq B_{\max}, \quad L(\Pi) \leq L_{\max},$$

где:

- $B(\Pi)$ — требуемая пропускная способность;
- $L(\Pi)$ — задержка реакции;
- $B_{\max}$ — допустимое ограничение пропускной способности сети;
- $L_{\max}$ — допустимое ограничение задержки реакции.

В общем виде задача синхронизации представляет собой многокритериальную оптимизацию:

$$\min_{\Pi} (\alpha \cdot D + \beta \cdot B + \gamma \cdot L),$$

где:

- α — вес важности согласованности;

- β — вес важности трафика;
- γ — вес важности задержки.

Для разрабатываемого метода важно, что минимизация рассогласования должна достигаться не только за счет более частой передачи данных. Увеличение частоты сообщений повышает нагрузку на сеть и не устраняет проблему нарушения порядка событий. Поэтому требуется метод, который ограничивает множество допустимых переходов состояния и обеспечивает корректную обработку событий даже при ненадежной доставке сообщений.

Формализованная постановка задачи представлена в виде IDEF0-диаграммы на рисунке 1.1.



Рисунок 1.1 – IDEF0-диаграмма постановки задачи обеспечения согласованности состояний

1.2.5 Требования к алгоритмам синхронизации

Алгоритмы, обеспечивающие согласованность игровых состояний, должны удовлетворять следующим требованиям:

- точность — алгоритм должен обеспечивать минимальные расхождения между состояниями, чтобы игроки наблюдали корректную картину происходящего;
- стабильность — изменения состояния и корректировки должны происходить без резких скачков и визуальных искажений;

- устойчивость к сетевым ошибкам — система должна корректно работать при задержках, потере отдельных пакетов, дублировании и нарушении порядка доставки;
- эффективность — алгоритм не должен перегружать вычислительные ресурсы и должен использовать ограниченный объем сетевого трафика;
- проверяемость переходов — система должна иметь возможность определить, допустимо ли входное событие в текущем состоянии;
- воспроизводимость обработки — одинаковая последовательность событий должна приводить к одинаковому результату на всех участниках взаимодействия.

Задача обеспечения согласованности игровых данных формализуется как задача управления состоянием распределенной асинхронной системы, в которой необходимо уравновесить точность, задержку реакции, сетевую нагрузку и корректность обработки событий.

1.3 Обзор существующих подходов к синхронизации состояния

Механизмы синхронизации состояния определяют, каким образом сервер и клиенты обеспечивают актуальность игрового мира, минимизируя расхождения между расчетами на стороне сервера и восприятием игрока. Ниже рассмотрены основные методы, используемые в современных сетевых играх различных жанров.

1.3.1 Периодическая репликация состояния

При использовании снимков сервер с фиксированной частотой отправляет клиентам актуальное состояние мира или его части [4]. Снимок включает координаты и характеристики активных объектов, состояние игровых сущностей и флаги игровой логики. Этот подход используется в шутерах и других динамичных играх, где важна высокая частота обновления положения объектов и событий [5].

Преимуществами периодической репликации состояния являются простота реализации и высокая точность согласования при стабильной сети.

Клиент регулярно получает авторитетные данные и может восстанавливать свое представление мира даже после потери отдельного обновления.

Недостатками подхода являются высокий объем трафика при большом числе объектов, возможные скачки состояния при применении обновлений и чувствительность к потере последовательных пакетов. Кроме того, передача снимков состояния не описывает причин перехода из одного состояния в другое. Поэтому данный подход хуже подходит для анализа логически значимых событий, которые должны быть применены строго один раз и в определенном порядке.

1.3.2 Репликация только изменений

При репликации изменений сервер отправляет не полный снимок, а только параметры, которые изменились со времени последнего известного клиенту состояния [6]. Данный метод применяется в системах, где полный снимок мира передавать нецелесообразно из-за большого количества объектов.

Преимуществами метода являются снижение объема передаваемых данных и эффективная работа в больших сценах. Если изменения затрагивают небольшую часть объектов, передача только измененных параметров существенно уменьшает сетевую нагрузку.

К недостаткам относится необходимость хранить предыдущее состояние, относительно которого вычисляется изменение. При потере сообщения клиент может оказаться в состоянии, к которому последующая дельта неприменима. В результате возможно накопление ошибок, поэтому такие системы обычно нуждаются в периодической отправке полного состояния для восстановления согласованности.

1.3.3 Событийная модель

В рамках событийной модели участникам передаются сведения не о полном состоянии объектов, а о произошедших событиях [7]. Событие описывает действие, которое должно быть применено к текущему состоянию: начало движения, взаимодействие с объектом, изменение режима, столкновение, создание или уничтожение сущности.

Событийная модель часто используется в RPG, МОБА и стратегиях, где значительная часть взаимодействий описывается дискретными событиями.

ми. Преимуществами подхода являются малый объем передаваемых данных, высокая эффективность для логических действий и совместимость с детерминированной симуляцией.

Главная сложность событийной модели заключается в необходимости одинаковой обработки событий на всех сторонах взаимодействия. Если событие потеряно, обработано повторно или применено в неправильном состоянии, локальные представления мира расходятся. Поэтому для событийной синхронизации особенно важны контроль порядка, идентификаторы событий, подтверждения доставки и проверка допустимости переходов.

Данный подход является наиболее близким к теме работы, так как конечный автомат может быть использован для формального описания того, какие события допустимы в каждом состоянии и к каким переходам они приводят.

1.3.4 Синхронизация по входным данным

При синхронизации по входным данным сервер или другой координирующий компонент рассылает клиентам не состояние, а входные команды игроков. Детерминированная симуляция на всех системах должна приводить к одинаковому результату [8].

Этот подход характерен для стратегий в реальном времени, файтингов и симуляторов, где критично поддерживать строгий детерминизм. Преимуществами являются минимальное сетевое потребление и высокая согласованность при условии, что симуляция действительно детерминирована.

Недостатки связаны со сложностью обеспечения детерминизма. Разные процессоры, разные реализации операций с плавающей точкой, отличия в частоте кадров и порядок обновления объектов могут привести к несовпадению результатов. Кроме того, необходимо синхронизировать тики симуляции и очереди входных команд, что повышает сложность реализации.

1.3.5 Предсказание состояния на клиенте

При клиентском предсказании клиент применяет действие игрока немедленно после ввода, не ожидая ответа сервера [9]. После получения авторитетных данных выполняется проверка и, при необходимости, корректировка состояния.

Механизм предсказания широко используется в динамичных шутерах и экшен-играх, требующих минимальной задержки отклика на ввод игрока. Преимуществами являются высокая отзывчивость управления и снижение субъективного влияния сетевой задержки.

Недостатками являются возможные рывки при корректировке, риск накопления ошибок при нестабильной сети и усложнение логики восстановления состояния. Для логически значимых событий предсказание особенно рискованно, так как ошибка может затронуть не только координаты объекта, но и состояние всей системы взаимодействия.

1.3.6 Откат сетевого кода

Метод отката используется в играх с высокой чувствительностью к синхронизации действий. Клиент предсказывает действия другого участника, а после получения реальных данных выполняется откат состояния и повторный расчет симуляции [10]. Rollback-подход применяется в аркадных PvP-играх и проектах с высокой чувствительностью к точности столкновений.

Преимуществами метода являются высокая точность взаимодействий и уменьшение видимого влияния сетевой задержки. Недостатками являются повышенная нагрузка на вычислительные ресурсы, необходимость хранить историю состояний и возможные визуальные искажения при пересчете.

Для разрабатываемого метода rollback-подход важен как пример обработки сетевой асинхронности через историю состояний. Однако в рассматриваемой задаче основной акцент делается не на пересчете физической симуляции, а на управлении допустимыми переходами дискретных состояний с помощью конечного автомата.

1.4 Обзор методов передачи игровых данных

1.4.1 Надежная и ненадежная доставка данных

В сетевых играх используются два основных подхода к передаче пакетов: надежная доставка и доставка без гарантий получения [11]. Они различаются по задержке, стабильности и объему служебной информации.

Надежная доставка, основанная на протоколе TCP, обеспечивает контроль целостности данных и сохраняет порядок передачи пакетов, повторно

отправляя утраченные сегменты [12]. Такая модель удобна для передачи данных, потеря которых недопустима. Однако TCP добавляет задержку из-за механизма подтверждений, повторных передач и сохранения общего порядка байтового потока. Для динамичных игр с высокой частотой обновления состояния это может приводить к задержке обработки новых данных из-за ожидания восстановления старых сегментов.

Ненадежная доставка, реализованная через протокол UDP, не гарантирует получение каждого отдельного пакета, но обеспечивает меньшие накладные расходы и минимально возможную задержку. Большинство сетевых игр используют UDP и собственные механизмы надежности для критичных данных. Такой подход позволяет выбирать, какие сообщения требуют подтверждения и повторной отправки, а какие могут быть потеряны без нарушения логики.

Для разрабатываемого метода использование UDP означает, что контроль доставки, порядка и дублирования логически значимых событий должен выполняться на прикладном уровне. Это соответствует событийной природе метода: не все данные требуют надежной передачи, но события, приводящие к переходам конечного автомата, должны быть обработаны согласованно.

1.4.2 Классификация и приоритизация игровых пакетов

Игровые данные по-разному реагируют на задержки и потери, поэтому пакеты обычно разделяют на классы приоритетности. Это позволяет оптимизировать передачу и избежать перегрузки канала связи.

Критичные данные включают команды игрока, результаты столкновений и важные события игровой логики. Они должны доставляться максимально быстро и целостно, поскольку их потеря может привести к заметной рассинхронизации состояния.

Важные, но допускающие потерю данные могут передаваться чаще, но без гарантии получения. Потеря части таких пакетов компенсируется новыми обновлениями в следующих сообщениях. Примером являются часто обновляемые координаты объектов, когда более новое значение делает устаревшее сообщение неактуальным.

Второстепенные данные, например визуальные эффекты, косметические параметры и отдельные звуковые события, могут передаваться с низким

приоритетом или не передаваться вовсе, если клиент способен корректно воспроизводить их локально.

В контексте данной работы к критичным данным относятся события, изменяющие состояние конечного автомата. Такие события должны иметь идентификатор, номер последовательности и признак подтверждения, чтобы принимающая сторона могла обнаружить потерю, повтор или нарушение порядка.

1.4.3 Механизмы обеспечения частичной надежности

Поскольку UDP сам по себе не предоставляет гарантированную доставку, игровые движки внедряют дополнительные механизмы частичной надежности, позволяющие гарантировать передачу важных данных без превращения всего обмена в аналог TCP.

К таким механизмам относятся:

- подтверждение доставки для отдельных типов пакетов;
- повторная отправка важных данных при отсутствии подтверждения;
- нумерация пакетов для восстановления порядка исполнения команд;
- фильтрация дубликатов по идентификаторам сообщений;
- отбрасывание устаревших сообщений, которые относятся к уже пройденному состоянию системы.

Эти механизмы позволяют экономно расходовать сетевой ресурс, сохраняя критичные данные в актуальном состоянии. При этом важно, чтобы повторная отправка не приводила к повторному применению одного и того же события. Поэтому прикладной уровень должен различать доставку сообщения и применение события к состоянию системы.

1.4.4 Адаптивные алгоритмы передачи данных

Современные сетевые игры используют динамические методы, которые подстраиваются под качество канала. Адаптивные алгоритмы позволяют снижать нагрузку в условиях высокой задержки или потери пакетов и увеличивать точность передачи при стабильном соединении.

К распространенным техникам относятся:

- изменение частоты отправки пакетов в зависимости от текущей задержки;
- снижение точности квантования при перегрузке сети;
- динамическое отключение второстепенных данных;
- регулирование частоты отправки обновлений в зависимости от важности объектов.

Такие методы повышают устойчивость сетевой игры при различных условиях соединения и снижают риск возникновения рассинхронизации. Однако они не заменяют механизмы согласования событий. Если логически значимое событие потеряно или обработано в неверном состоянии, уменьшение частоты передачи второстепенных данных не восстановит корректность системы.

1.5 Типовые игровые данные, подлежащие хранению, обновлению и передаче

В процессе работы многопользовательского игрового приложения сервер и клиенты обмениваются различными типами данных, описывающих состояние объектов, игровые события и вспомогательные параметры [13]. Эти данные определяют внутреннюю логику игрового мира и корректность взаимодействия игроков.

1.5.1 Данные состояния объектов

Состояние объектов описывает характеристики сущностей и является основой игрового мира. Эти данные подлежат частому обновлению.

К типичным данным состояния объектов относятся:

- позиция объекта;
- скорость и направление движения;
- поворот и ориентация;

- состояние анимации;
- физические параметры.

Эти данные требуют высокой точности и частой синхронизации. Особенно это критично для FPS, гонок и экшен-игр, где любое отклонение в позиции приводит к заметным ошибкам восприятия. В то же время часть таких данных может устаревать: если новое значение позиции уже получено, старое значение обычно не требуется применять.

1.5.2 Данные игровой логики

Помимо визуального состояния объектов существуют параметры, влияющие на игровую логику.

В эту категорию входят:

- здоровье;
- ресурсы;
- активные эффекты;
- флаги состояний;
- текущая фаза взаимодействия.

Данные игровой логики обновляются реже, чем параметры движения, но требуют более надежной доставки. Ошибка в таких данных приводит к различию в результатах игровых событий. Например, если один участник считает, что действие уже завершено, а другой продолжает считать его активным, то последующие события будут интерпретированы по-разному.

1.5.3 Данные взаимодействия и событий

Игровые события отображают дискретные изменения состояния мира. Они могут быть результатом действий игрока или следствием внутренних механик.

Примеры событий:

- совершение действия;

- взаимодействие с объектом;
- создание или уничтожение объекта;
- начало или завершение игровой фазы;
- срабатывание триггера.

События требуют надежной обработки, поскольку их потеря приводит к расхождению в симуляции между игроками. Кроме того, событие должно применяться в определенном состоянии системы. Если событие доставлено позже, чем система перешла в следующее состояние, оно может стать устаревшим и должно быть обработано особым образом.

Именно данные взаимодействия и событий являются основным объектом рассмотрения в разрабатываемом методе. Для них необходимо обеспечить доставку, порядок, защиту от повторного применения и проверку допустимости перехода.

1.5.4 Вспомогательные данные

Некоторые игры используют дополнительные данные, не относящиеся напрямую к состоянию объектов, но влияющие на корректность симуляции.

К таким данным относятся:

- навигационные сетки;
- вспомогательные маркеры;
- данные о временных параметрах;
- параметры конфигурации игровой сессии.

Эти данные чаще хранятся на сервере и передаются клиенту при необходимости. Их изменение обычно происходит реже, чем изменение состояния объектов, однако несогласованность таких данных может привести к разному поведению симуляции на разных сторонах.

1.5.5 Визуальные данные

Некоторые данные не влияют на игровую логику напрямую и могут синхронизироваться реже или генерироваться на клиенте без участия сервера.

К таким данным относятся:

- эффекты частиц;
- косметические параметры;
- звуковые события;
- вспомогательные элементы интерфейса.

Передача таких данных оптимизируется в первую очередь при ограничении пропускной способности. Потеря визуального события может ухудшить восприятие, но обычно не приводит к нарушению логики взаимодействия.

1.5.6 Метаданные и технические данные

Помимо игровых данных существует набор технической информации, обеспечивающей корректную работу сетевого взаимодействия.

К ним относятся:

- идентификаторы объектов;
- временные метки;
- номера последовательности;
- контрольные суммы;
- служебные флаги;
- идентификаторы подтверждений доставки.

Такие данные не видны игроку, но они важны для поддержания согласованности состояния и предотвращения ошибок. В методе, основанном на конечных автоматах, технические данные используются для связи сетевого сообщения с конкретным событием, состоянием автомата и фактом обработки этого события.

1.6 Основные причины возникновения рассинхронизации

Рассинхронизация состояния — это расхождение между данными, хранящимися на авторитетной стороне, и состоянием, которое отображается или обрабатывается в клиентских приложениях. Возникновение рассинхронизации обусловлено как особенностями сетевого окружения, так и архитектурными проблемами внутри игрового приложения.

1.6.1 Сетевые задержки

Одной из наиболее распространенных причин рассинхронизации является задержка передачи данных [14]. Игровой клиент всегда получает состояние с определенной задержкой, зависящей от качества соединения. Если задержка непостоянна и резко меняется, клиент может получить обновления позже или раньше ожидаемого момента, что вызывает временные расхождения в позициях объектов и фазах событий.

Ситуация ухудшается в условиях нестабильных сетей, где вариативность задержек возрастает. Для событийной синхронизации это означает, что событие, созданное раньше, может быть доставлено позже события, созданного после него. Без контроля порядка такая ситуация приводит к некорректной последовательности переходов состояния.

1.6.2 Потеря, дублирование и нарушение порядка доставки пакетов

При использовании ненадежных протоколов часть пакетов может теряться или приходить в другом порядке. Это приводит к следующим ошибкам:

- объект остается на устаревшей позиции;
- проигрываются неактуальные события;
- происходит откат состояния назад;
- одно и то же событие применяется несколько раз;
- переход состояния выполняется без необходимого предшествующего события.

Хотя игровые движки реализуют механизмы частичной надежности, абсолютной защиты от подобных проблем нет. Поэтому прикладная логика должна быть готова к получению некорректных, повторных и устаревших сообщений.

1.6.3 Недетерминизм симуляции

Некоторые игры опираются на локальный расчет физики или игровой логики на стороне клиента. Любое незначительное отличие в вычислениях может привести к расхождению состояния. Основные источники недетерминизма:

- использование чисел с плавающей точкой, дающих разные результаты на разных процессорах;
- различный порядок обновления объектов на разных клиентах;
- вариации частоты кадров, влияющие на расчет физики.

Такие проблемы особенно заметны в стратегиях и симуляторах, где большое количество объектов должно развиваться по одинаковым правилам в течение длительного времени. В задачах с конечными автоматами недетерминизм проявляется как неодинаковое выполнение переходов при одинаковых входных событиях, что делает систему непредсказуемой.

1.6.4 Различия в производительности клиентских устройств

Игровые клиенты могут работать с разной скоростью в зависимости от мощности процессора, видеокарты и текущей нагрузки. Если частота обновления симуляции отличается, одни клиенты рассчитывают больше физических шагов, чем другие, а предсказания состояния расходятся с авторитетной логикой.

В играх, где симуляция сильно зависит от стабильной частоты кадров, такие различия являются источником ошибок. Для снижения влияния этого фактора часто используется фиксированный шаг симуляции, синхронизация тиков и отделение логического состояния от частоты отрисовки.

1.6.5 Неправильная или неполная сериализация данных

При передаче состояния объекты сериализуются. Ошибки сериализации приводят к тому, что данные не совпадают на клиенте и сервере. Это может быть вызвано:

- пропуском некоторых параметров объекта;
- отправкой данных в неправильном формате;
- несогласованностью типов при чтении и записи;
- различием версий протокола обмена.

Для событийной модели ошибка сериализации особенно опасна, если затрагивает идентификатор события, номер последовательности, тип события или состояние, к которому оно относится. В этом случае сообщение может быть доставлено, но обработано как другое событие или как событие для другого состояния.

1.6.6 Несогласованный порядок выполнения событий

Если события обрабатываются не в том же порядке, что на авторитетной стороне, результаты симуляции расходятся. Это происходит из-за локальных оптимизаций клиента, попадания пакетов с разными временными метками, параллельного выполнения задач и применения предсказания до получения подтверждения от сервера.

Несогласованная обработка событий особенно критична для игр с динамичными механиками взаимодействия. Например, если событие завершения действия обработано раньше события его начала, система может перейти в недопустимое состояние. Поэтому порядок событий должен контролироваться не только на транспортном уровне, но и на уровне модели состояний.

1.6.7 Ошибки в предсказании и последующей коррекции состояния

Клиентское предсказание значительно повышает отзывчивость, но несет риск ошибок. Предсказание может расходиться с реальными данными сервера.

ра, что приводит к резким скачкам при корректировке, неверным расчетам следующего состояния и накоплению ошибки при плохой связи [15].

Такие проблемы заметны при нестабильной задержке сети и большом количестве корректировок. В дискретной логике взаимодействия ошибка предсказания может привести к тому, что клиент временно перейдет в состояние, которое затем будет отклонено авторитетной стороной. В этом случае система должна иметь способ безопасно отменить или скорректировать переход.

Вывод

В данном разделе была рассмотрена предметная область синхронизации игровых состояний в многопользовательских приложениях. Было показано, что такие приложения являются распределенными асинхронными системами, в которых рассинхронизация может возникать из-за задержек, потерь пакетов, нарушения порядка доставки, недетерминизма симуляции и неодинаковой обработки событий.

Была формализована задача обеспечения согласованности игровых данных. Состояние игрового мира представлено как совокупность состояний сущностей, изменяемых под воздействием входных команд и событий. Рас согласование состояния определено через различие параметров сущностей и логических состояний системы.

2 Конструкторский раздел

2.1 Требования к разрабатываемому методу

Разрабатываемый метод должен обеспечивать согласованную обработку логически значимых игровых событий в условиях асинхронного сетевого обмена. Под логически значимым событием понимается событие, приводящее к изменению состояния игрока, общего игрового объекта или состояния взаимодействия между двумя игроками.

К методу синхронизации предъявляются следующие требования:

- метод должен описывать взаимодействие двух игроков как обработку событий, вызывающих допустимые переходы конечного автомата синхронизируемого объекта;
- обработка критичных событий должна выполняться через авторитетный сервер, проверяющий порядок сообщений и допустимость применения события;
- каждое событие, изменяющее состояние системы, должно иметь уникальный идентификатор;
- каждое событие должно иметь номер последовательности в потоке событий игрока-отправителя;
- повторное получение одного и того же события не должно приводить к повторному изменению состояния;
- потеря события или подтверждения должна обнаруживаться на прикладном уровне;
- неподтвержденные события должны повторно отправляться до получения подтверждения;
- события, недопустимые для целевого конечного автомата, должны отклоняться;
- метод должен учитывать возможность задержки, потери, дублирования и нарушения порядка доставки UDP-пакетов.

Ограничением метода является рассмотрение взаимодействия двух игроков. Такое ограничение позволяет явно описать события и состояния игровых объектов, не вводя дополнительные механизмы масштабирования.

2.2 Функциональная модель метода

Разрабатываемый метод использует клиент-серверную схему обмена. Клиент формирует событие по локальному действию игрока и отправляет его серверу по UDP. Сервер выступает авторитетной стороной. Он проверяет событие, применяет его к целевому конечному автомату, отправляет подтверждение игроку-отправителю и рассылает подтвержденное событие клиентам.

Метод синхронизации включает следующие основные этапы:

1. формирование локального игрового события на стороне клиента;
2. присвоение событию идентификатора и номера последовательности;
3. отправка события серверу;
4. сохранение события в журнале неподтвержденных событий клиента;
5. получение события сервером;
6. проверка идентификатора события и фильтрация дубликатов;
7. проверка номера последовательности события;
8. поиск целевого объекта по идентификатору;
9. применение события к конечному автомату целевого объекта;
10. отправка подтверждения клиенту-отправителю;
11. рассылка подтвержденного события клиентам;
12. удаление подтвержденного события из журнала неподтвержденных событий клиента.

Декомпозиция функции обеспечения согласованности состояний приведена на рисунке 2.1. Она отражает последовательность обработки события

от формирования на стороне клиента до подтверждения обработки и распространения результата.

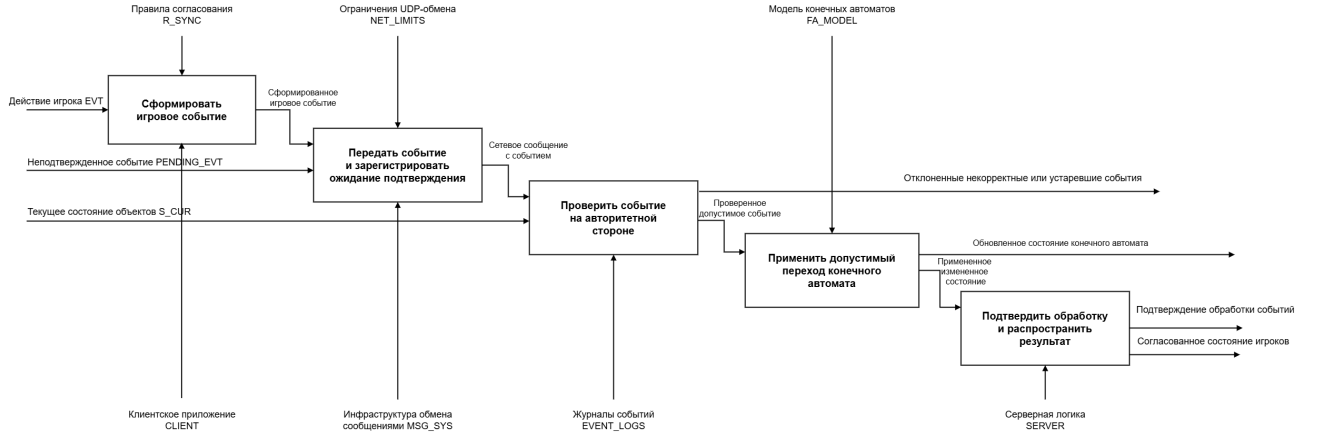


Рисунок 2.1 – IDEF0-диаграмма метода обеспечения согласованности состояний

2.3 Формальная модель синхронизации

Состояние системы двух игроков в момент времени t зададим как:

$$S(t) = (S_1(t), S_2(t), O(t), N(t)) ,$$

где:

- $S_1(t)$ — локальное состояние первого игрока;
- $S_2(t)$ — локальное состояние второго игрока;
- $O(t)$ — множество состояний синхронизируемых игровых объектов;
- $N(t)$ — состояние сетевого протокола синхронизации.

Множество синхронизируемых игровых объектов зададим как:

$$O = \{o_1, o_2, \dots, o_n\}.$$

Каждый объект o_j имеет идентификатор $id(o_j)$ и конечный автомат:

$$A_j = (Q_j, E_j, \delta_j, q_j^0) ,$$

где:

- Q_j — множество состояний объекта;
- E_j — множество допустимых событий объекта;
- δ_j — функция переходов;
- q_j^0 — начальное состояние.

Игровое событие зададим кортежем:

$$e_k = (eventId_k, sequence_k, playerId_k, sourceObject_k, targetObject_k, type_k),$$

где:

- $eventId_k$ — уникальный идентификатор события;
- $sequence_k$ — номер события в потоке игрока-отправителя;
- $playerId_k$ — идентификатор игрока-отправителя;
- $sourceObject_k$ — идентификатор объекта-источника;
- $targetObject_k$ — идентификатор целевого объекта;
- $type_k$ — тип события.

Сетевое сообщение, передаваемое по UDP, зададим кортежем:

$$m_k = (messageType_k, event_k, ackId_k, senderId_k, targetId_k),$$

где:

- $messageType_k$ — тип сетевого сообщения;
- $event_k$ — игровое событие;
- $ackId_k$ — идентификатор подтверждаемого события;
- $senderId_k$ — идентификатор отправителя сообщения;
- $targetId_k$ — идентификатор получателя подтверждения.

В методе используются три типа сетевых сообщений:

- **ClientHello** — служебное сообщение регистрации клиента;
- **PuzzleEvent** — сообщение, содержащее игровое событие;
- **Acknowledgement** — подтверждение обработки события.

Состояние протокола синхронизации на стороне клиента можно представить кортежем:

$$N_c = (id_c, seq_c^{out}, L_c, A_c),$$

где:

- id_c — идентификатор клиента;
- seq_c^{out} — номер следующего исходящего события;
- L_c — журнал отправленных, но не подтвержденных событий;
- A_c — множество уже примененных событий.

Состояние протокола синхронизации на стороне сервера можно представить кортежем:

$$N_s = (R_s, A_s, H_s),$$

где:

- R_s — множество зарегистрированных конечных автоматов игровых объектов;
- A_s — множество уже примененных событий;
- H_s — отображение идентификатора игрока в номер следующего ожидаемого события.

Применение события к игровому объекту определяется функцией:

$$Apply(e_k, o_j) = \begin{cases} true, & \text{если } type_k \in E_j \text{ и переход } \delta_j \text{ допустим;} \\ false, & \text{если событие не может быть применено к объекту.} \end{cases}$$

В таком представлении сетевой уровень передает события, менеджер синхронизации проверяет идентификаторы и порядок сообщений, а допустимость перехода определяется целевым объектом, реализующим конечный автомат.

2.4 Состояния протокола синхронизации

Для описания работы метода выделим следующие состояния протокола синхронизации:

- q_0 — регистрация клиента;
- q_1 — ожидание локального или сетевого события;
- q_2 — формирование исходящего события;
- q_3 — ожидание подтверждения;
- q_4 — обработка события на сервере;
- q_5 — рассылка подтвержденного события;
- q_6 — применение подтвержденного события на клиенте;
- q_7 — повторная отправка неподтвержденного события;
- q_8 — отклонение события.

Состояние q_0 используется при запуске клиента. Клиент отправляет серверу сообщение `ClientHello`, содержащее идентификатор игрока. Сервер сохраняет сетевой адрес клиента для последующей рассылки сообщений.

Состояние q_1 соответствует обычному режиму работы, в котором клиент ожидает локального действия игрока или входящего сетевого сообщения.

Состояние q_2 возникает при локальном действии игрока. Действие преобразуется в событие, которому присваиваются идентификатор и номер последовательности.

Состояние q_3 используется после отправки критичного события серверу. Клиент хранит событие в журнале неподтвержденных событий и ожидает подтверждения.

Состояние q_4 соответствует обработке события сервером. Сервер проверяет дубликаты, номер последовательности, наличие целевого объекта и допустимость применения события к конечному автомату.

Состояние q_5 используется после успешного применения события сервером. Сервер отправляет подтверждение игроку-отправителю и рассылает подтвержденное событие клиентам.

Состояние q_6 соответствует обработке подтвержденного события клиентом. Клиент применяет событие к локальному конечному автомату целевого объекта, если событие еще не было обработано.

Состояние q_7 возникает при истечении времени ожидания подтверждения. Клиент повторно отправляет событие с теми же значениями идентификатора и номера последовательности.

Состояние q_8 используется при получении дубликата, устаревшего события, события с нарушенным порядком или события, которое не может быть применено к целевому конечному автомату.

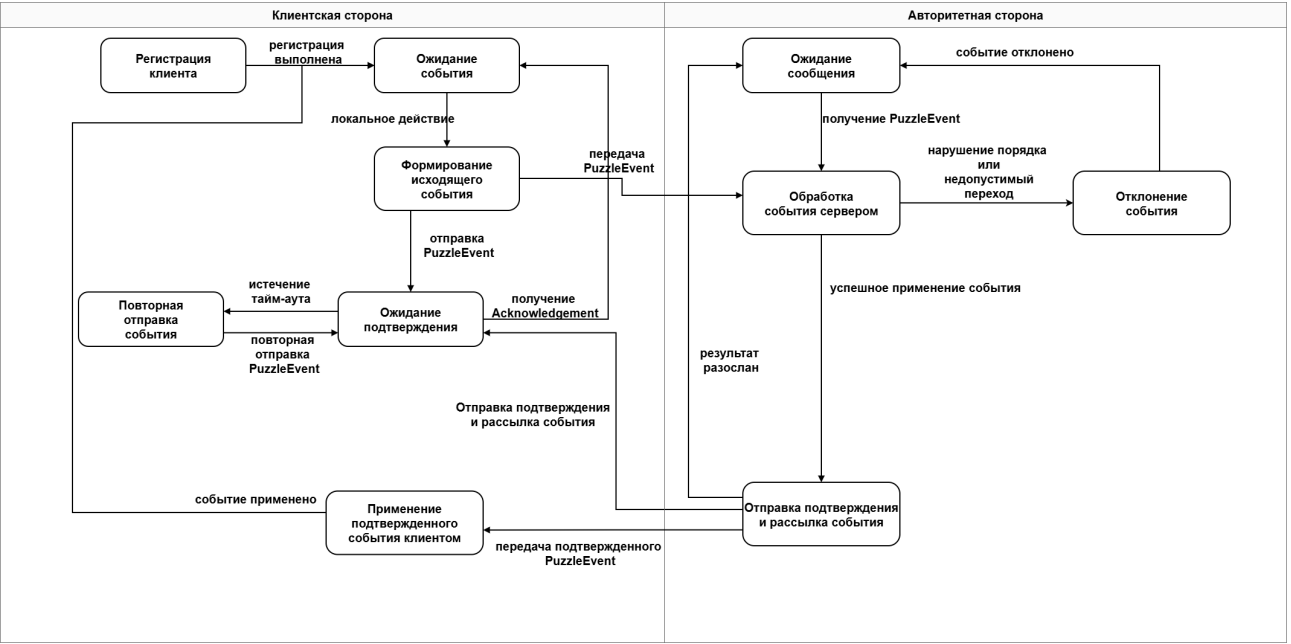


Рисунок 2.2 – Диаграмма состояний протокола синхронизации

2.5 Модель переходов протокола синхронизации

Основные переходы протокола синхронизации приведены в таблице 2.1.

Таблица 2.1 – Основные переходы протокола синхронизации

Текущее состояние	Событие	Условие	Следующее состояние
Регистрация клиента	Отправлен ClientHello	Клиент имеет идентификатор	Ожидание события

Текущее состояние	Событие	Условие	Следующее состояние
Ожидание события	Локальное действие игрока	Сформирован тип события	Формирование исходящего события
Формирование исходящего события	Событие отправлено серверу	Событие сохранено в журнале	Ожидание подтверждения
Ожидание подтверждения	Получен Acknowledgement	Идентификатор события совпадает	Ожидание события
Ожидание подтверждения	Истек тайм-аут	Событие остается в журнале	Повторная отправка
Повторная отправка	Событие отправлено повторно	Идентификатор и номер не изменены	Ожидание подтверждения
Ожидание события	Получен PuzzleEvent сервером	Номер события ожидаемый	Обработка события сервером
Обработка события сервером	Событие применено к FSM	Переход допустим	Рассылка подтвержденного события
Обработка события сервером	Событие является дубликатом	Ключ события уже обработан	Отклонение события
Обработка события сервером	Нарушен порядок	Номер больше ожидаемого	Отклонение события

Текущее состояние	Событие	Условие	Следующее состояние
Рассылка подтвержденного события	Событие отправлено клиентам	Событие принято сервером	Ожидание события
Ожидание события	Получено подтвержденное событие клиентом	Событие не обработано ранее	Применение события клиентом
Применение события клиентом	Событие применено к FSM	Переход допустим	Ожидание события

Дубликат события не должен приводить к повторному переходу конечного автомата. Если ключ события уже содержится во множестве обработанных событий, состояние объекта не изменяется. На сервере при получении повторного события дополнительно отправляется подтверждение игроку-отправителю, поскольку повтор мог возникнуть из-за потери предыдущего подтверждения.

Если сервер получает событие с номером последовательности меньше ожидаемого, оно рассматривается как устаревшее. Если номер больше ожидаемого, событие не применяется, так как перед ним должно быть обработано одно или несколько предыдущих событий. Такое сообщение не подтверждается, что приводит к повторной отправке со стороны клиента до восстановления корректного порядка доставки.

2.6 Структура передаваемых событий

Передаваемые сообщения делятся на служебные сообщения регистрации, игровые события и подтверждения. Игровые события изменяют состояние конечного автомата целевого игрового объекта. Подтверждения используются для удаления события из журнала неподтвержденных событий клиента.

К игровым событиям относятся:

- нажатие интерактивной кнопки;
- нажатие или отпускание нажимной плиты;

- переключение рычага;
- активация моста;
- нажатие цветной кнопки;
- завершение игровой задачи.

Структура игрового события должна включать поля, необходимые для маршрутизации события, контроля порядка и защиты от повторной обработки. В таблице 2.2 приведен набор полей события.

Таблица 2.2 – Структура игрового события

Поле	Назначение
<code>eventId</code>	Уникальный идентификатор события
<code>sequenceNumber</code>	Номер события в потоке игрока-отправителя
<code>sourcePlayerId</code>	Идентификатор игрока-отправителя
<code>sourceObjectId</code>	Идентификатор объекта, сформировавшего событие
<code>targetObjectId</code>	Идентификатор объекта, к которому должно быть применено событие
<code>eventType</code>	Тип игрового события

Структура сетевого сообщения приведена в таблице 2.3.

Таблица 2.3 – Структура сетевого сообщения

Поле	Назначение
<code>messageType</code>	Тип сетевого сообщения: <code>ClientHello</code> , <code>PuzzleEvent</code> или <code>Acknowledgement</code>
<code>puzzleEvent</code>	Игровое событие, передаваемое в сообщении типа <code>PuzzleEvent</code>
<code>ackForEventId</code>	Идентификатор подтверждаемого события
<code>senderPlayerId</code>	Идентификатор отправителя сообщения
<code>targetPlayerId</code>	Идентификатор клиента, которому адресовано подтверждение

Поле `sequenceNumber` используется для контроля порядка, а поле `eventId` вместе с `sourcePlayerId` — для фильтрации дубликатов. Допустимость перехода не передается в сообщении явно, она определяется целевым объектом при вызове метода применения события.

2.7 Механизм подтверждения и повторной отправки

Так как UDP не гарантирует доставку сообщений, надежность для критичных событий обеспечивается на прикладном уровне. После отправки события клиент помещает его в журнал неподтвержденных событий L_c . В журнале событие хранится по идентификатору `eventId`.

Сервер после успешной проверки и применения события отправляет подтверждение. Подтверждение содержит идентификатор принятого события и идентификатор клиента, которому оно адресовано. После получения подтверждения клиент удаляет событие из журнала неподтвержденных событий.

Если подтверждение не получено за время T_{ack} , клиент повторно отправляет событие. Повторная отправка не создает новое событие, а использует тот же идентификатор и тот же номер последовательности. Благодаря этому сервер может отличить повторную отправку от нового игрового события. В разработанном методе повторная отправка продолжается, пока событие находится в журнале неподтвержденных событий.

Последовательность обмена сообщениями в обоих случаях определяется состояниями протокола. После отправки `PuzzleEvent` клиент переходит в

ожидание подтверждения, сервер выполняет авторитетную проверку события, а при отсутствии подтверждения клиент повторно отправляет тот же экземпляр события.

2.8 Контроль порядка сообщений

Для каждого игрока сервер поддерживает номер следующего ожидаемого события. Клиент увеличивает номер последовательности при создании нового логически значимого события. Сервер сравнивает полученный номер с ожидаемым значением.

При получении события возможны следующие случаи:

- номер события равен ожидаемому — событие может быть проверено и применено;
- номер события меньше ожидаемого — событие считается устаревшим или повторным;
- номер события больше ожидаемого — одно или несколько предыдущих событий отсутствуют, поэтому текущее событие не применяется.

Для двух игроков достаточно хранить по одному счетчику ожидаемого события на каждого игрока. Это упрощает реализацию по сравнению с системами, где требуется поддерживать множество потоков событий от большого числа участников.

Если событие с большим номером последовательности приходит раньше предыдущих событий, сервер не применяет его и не отправляет подтверждение. Клиент продолжает повторную отправку неподтвержденного события, что позволяет восстановить обработку после доставки недостающего сообщения.

2.9 Алгоритм синхронизации состояний

Схемы алгоритмов обработки исходящего события, серверной обработки события и повторной отправки приведены на рисунках 2.3–2.5.

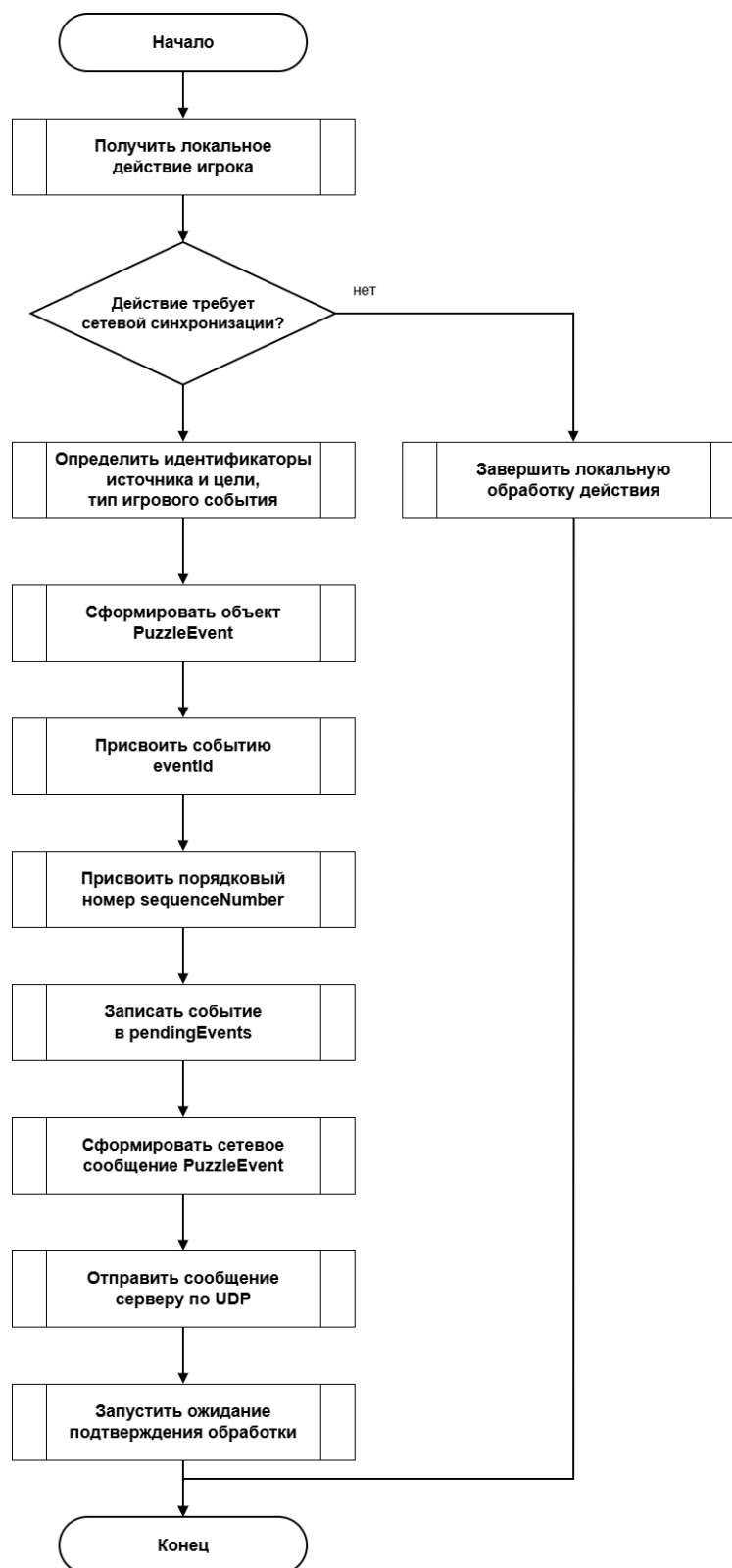


Рисунок 2.3 – Схема алгоритма обработки исходящего события

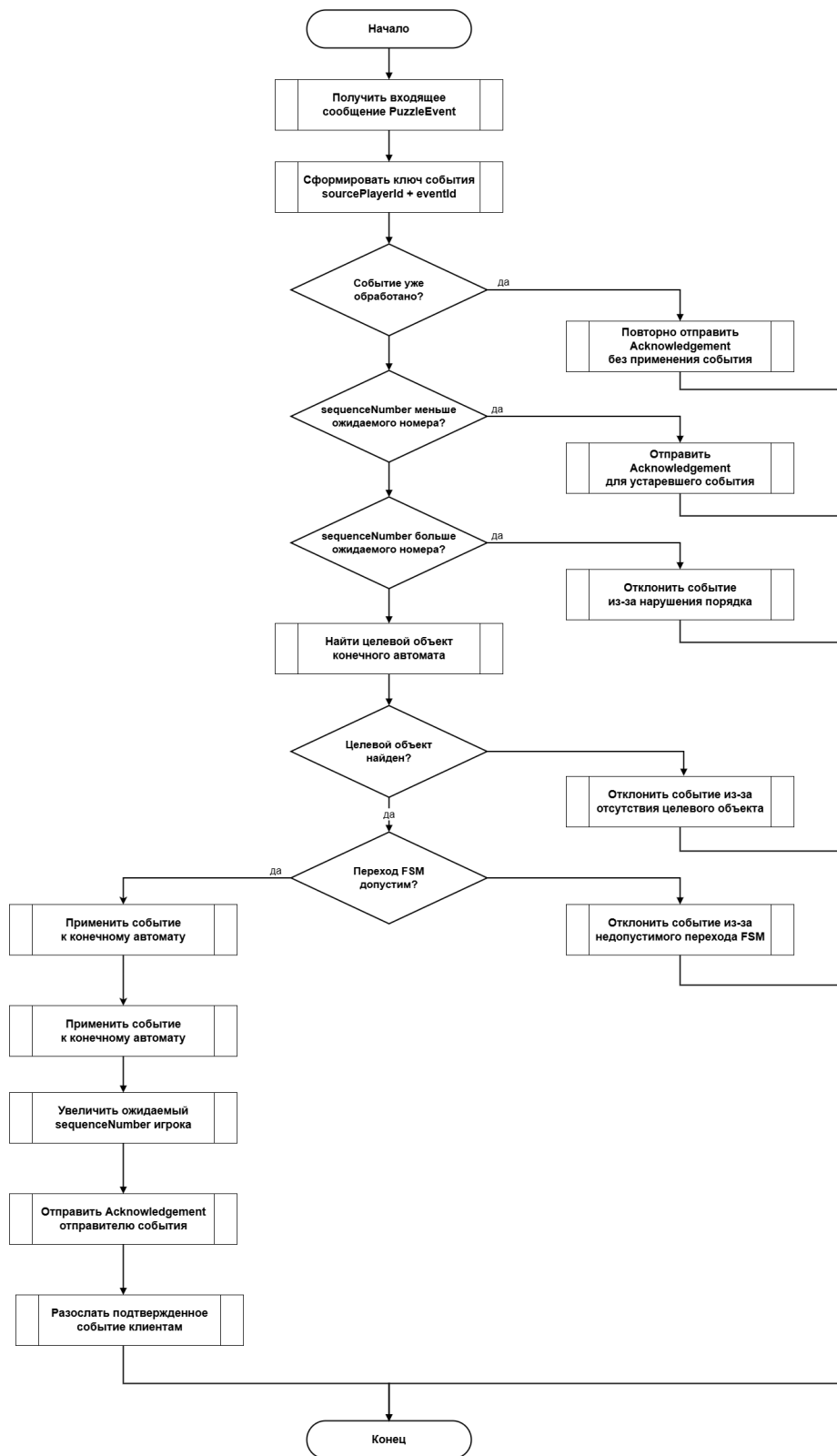


Рисунок 2.4 – Схема алгоритма обработки события сервером



Рисунок 2.5 – Схема алгоритма повторной отправки неподтвержденных событий

2.10 Структура программного обеспечения

Для реализации предложенного метода разрабатываемое программное обеспечение должно включать следующие компоненты:

- модуль сетевого обмена, выполняющий отправки и прием UDP-сообщений;

- модуль сериализации и десериализации сетевых сообщений;
- журнал неподтвержденных событий клиента;
- хранилище обработанных событий для фильтрации дубликатов;
- хранилище ожидаемых номеров последовательности на сервере;
- менеджер синхронизации событий;
- конечные автоматы игровых объектов;
- модель игрового состояния двух игроков.

Взаимодействие компонентов должно быть организовано так, чтобы сетевой модуль не изменял игровое состояние напрямую. Все входящие сообщения должны передаваться менеджеру синхронизации, который принимает решение о допустимости события и передает его целевому конечному автомату. Это разделяет транспортный уровень и уровень игровой логики, что снижает вероятность некорректного изменения состояния при ошибках сети.

Вывод

В данном разделе был разработан метод обеспечения согласованности состояний двух асинхронных игроков с использованием событийной модели и конечных автоматов. Были сформулированы требования к методу, определена клиент-серверная функциональная модель синхронизации, задана формальная модель состояния игроков, игровых объектов, событий и сетевых сообщений.

Была предложена структура протокола синхронизации, включающая регистрацию клиента, формирование события, ожидание подтверждения, обработку события сервером, рассылку подтвержденного события, применение события клиентом, повторную отправку и отклонение некорректных событий. Для модели были описаны основные переходы и условия их выполнения.

3 Технологический раздел

3.1 Выбор средств программной реализации

Для программной реализации метода была выбрана игровая среда Unity. Данный выбор обусловлен тем, что Unity предоставляет событийную модель выполнения сценариев, поддержку компонентов, механизм обработки пользовательского ввода, средства моделирования интерактивных сцен и возможность разработки логики на языке C# [16]. Это позволяет реализовать не только сетевой обмен, но и программное обеспечение для проверки взаимодействия двух игроков.

В качестве языка программирования выбран C#. Язык поддерживает объектно-ориентированную модель, обобщенные коллекции, перечисления, классы, интерфейсы и средства работы с сетевыми сокетами [17]. Эти свойства используются при реализации конечных автоматов, событий, журналов неподтвержденных сообщений и сетевого транспортного уровня.

Для сетевого обмена выбран протокол UDP. Выбор UDP соответствует постановке задачи, так как метод должен работать в условиях отсутствия встроенных гарантий доставки и порядка сообщений. В реализации используется класс `UdpClient` пространства имен `System.Net.Sockets`, предоставляющий средства отправки и приема UDP-датаграмм [18]. Прикладная надежность реализуется через подтверждения, повторную отправку, контроль идентификаторов событий и проверку порядка сообщений.

3.2 Структура программного обеспечения

Разработанное программное обеспечение состоит из нескольких групп модулей:

- модуль сетевого обмена;
- модуль синхронизации событий и конечных автоматов;
- модули объектов, реализующих интерфейс конечного автомата;
- модули пользовательского взаимодействия;
- модули сбора исследовательских метрик.

Основные классы программной реализации приведены в таблице 3.1.

Таблица 3.1 – Основные классы программной реализации

Класс	Назначение
<code>UdpEventTransport</code>	Отправка и прием UDP-сообщений, имитация потерь, задержки, вариативности задержки и дублирования пакетов
<code>NetworkMessage</code>	Представление сетевого сообщения, содержащего игровое событие, подтверждение или служебное сообщение подключения
<code>EventFsmSyncManager</code>	Центральный модуль синхронизации событий, обработки подтверждений, повторной отправки и применения событий к конечным автоматам
<code>PuzzleEvent</code>	Представление логически значимого игрового события
<code>IFsmPuzzleObject</code>	Интерфейс объектов, состояние которых изменяется через конечный автомат
<code>PuzzleDoor</code>	Пример объекта, реализующего интерфейс <code>IFsmPuzzleObject</code>
<code>ToggleBridge</code>	Пример объекта, реализующего интерфейс <code>IFsmPuzzleObject</code>
<code>CoopHoldPuzzleController</code>	Пример объекта, реализующего интерфейс <code>IFsmPuzzleObject</code> для составного состояния взаимодействия
<code>ColorSequenceController</code>	Пример объекта, реализующего интерфейс <code>IFsmPuzzleObject</code> для последовательной обработки событий
<code>ResearchMetrics</code>	Сбор статистики отправки сообщений, потерь, повторных отправок, отклоненных дубликатов и примененных событий

Структура программного обеспечения представлена в виде диаграммы классов на рисунке 3.1. Конкретные классы игровых объектов на диаграмме рассматриваются не как самостоятельная предметная область, а как реализации общего интерфейса `IFsmPuzzleObject`, через который метод применяет подтвержденные события к конечным автоматам.

Диаграмма классов программной реализации метода синхронизации

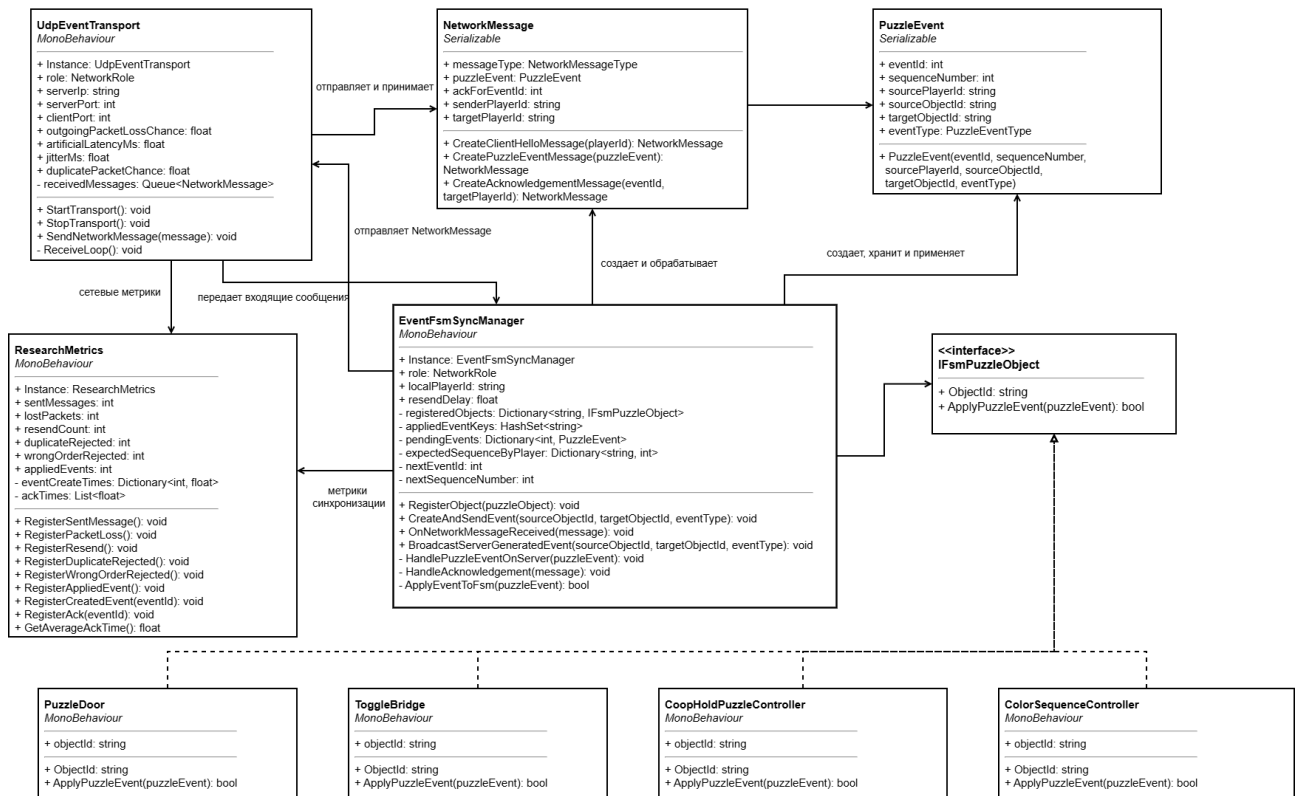


Рисунок 3.1 – Диаграмма классов программной реализации

3.3 Реализация сетевого обмена

Сетевой обмен реализован в классе `UdpEventTransport`. Объект данного класса может работать в одной из двух ролей: `Server` или `Client`. Роль задается перечислением `NetworkRole`. Сервер открывает UDP-сокеты на серверном порту и регистрирует адреса клиентов после получения служебного сообщения `ClientHello`. Клиент открывает собственный UDP-сокеты, хранит адрес сервера и после запуска отправляет сообщение подключения.

При отправке сообщение типа `NetworkMessage` сериализуется в JSON-строку. Затем строка преобразуется в массив байтов и передается через UDP. Перед непосредственной отправкой применяется модель сетевых условий, включающая:

- вероятность потери исходящего пакета;
- искусственную задержку отправки;
- вариативность задержки;

— вероятность дублирования пакета.

Прием UDP-сообщений выполняется в отдельном фоновом потоке. Это необходимо для того, чтобы ожидание входящих датаграмм не блокировало основной цикл Unity. Полученные сообщения помещаются в очередь `receivedMessages`. Доступ к очереди синхронизируется с помощью объекта блокировки. В методе `Update` сообщения извлекаются из очереди и передаются в `EventFsmSyncManager` уже в основном потоке Unity.

Игровые объекты, компоненты сцены и визуальное состояние должны изменяться из основного потока по правилам Unity. Поэтому фоновый поток отвечает только за сетевой прием, а изменение конечных автоматов и игровых объектов выполняется после передачи сообщения в основной цикл.

Фрагмент реализации отправки сообщения с учетом потерь, задержки, вариативности задержки и дублирования приведен в листинге 3.1.

Листинг 3.1 – Отправка UDP-сообщения с имитацией сетевых условий

```
1  public void SendNetworkMessage(NetworkMessage message)
2  {
3      if (message == null)
4      {
5          Debug.LogError("[UDP] Нельзя отправить пустое сообщение");
6          return;
7      }
8
9      string json = JsonUtility.ToJson(message);
10     byte[] data = Encoding.UTF8.GetBytes(json);
11
12     StartCoroutine(SendWithNetworkConditions(data, json,
13         ↪ message.messageType));
14
15     private IEnumerator SendWithNetworkConditions(byte[] data, string
16         ↪ json, NetworkMessageType messageType)
17     {
18         if (Random.value < outgoingPacketLossChance)
19         {
20             Debug.LogWarning("[UDP " + role + "] Имитируем потерю
21                 ↪ исходящего пакета: " + messageType);
```

Продолжение листинга 3.1

```
21         if (ResearchMetrics.Instance != null)
22         {
23             ResearchMetrics.Instance.RegisterPacketLoss();
24         }
25
26         yield break;
27     }
28
29     float finalDelayMs = artificialLatencyMs;
30
31     if (jitterMs > 0f)
32     {
33         finalDelayMs += Random.Range(-jitterMs, jitterMs);
34     }
35
36     if (finalDelayMs < 0f)
37     {
38         finalDelayMs = 0f;
39     }
40
41     if (finalDelayMs > 0f)
42     {
43         yield return new WaitForSeconds(finalDelayMs / 1000f);
44     }
45
46     SendRaw(data, json);
47
48     if (ResearchMetrics.Instance != null)
49     {
50         ResearchMetrics.Instance.RegisterSentMessage();
51     }
52
53     if (Random.value < duplicatePacketChance)
54     {
55         Debug.LogWarning("[UDP " + role + "] Имитируем дубликат
56             ↳ пакета: " + messageType);
57
58         yield return new WaitForSeconds(0.05f);
59
60         SendRaw(data, json);
```

Окончание листинга 3.1

```
60
61         if (ResearchMetrics.Instance != null)
62         {
63             ResearchMetrics.Instance.RegisterSendMessage();
64         }
65     }
66 }
```

Фрагмент приема сообщений в фоновом потоке и передачи их в основной поток Unity приведен в листинге 3.2.

Листинг 3.2 – Прием UDP-сообщений и обработка очереди в основном потоке

```
1     private void ReceiveLoop()
2     {
3         while (isRunning)
4         {
5             try
6             {
7                 IPEndPoint senderEndPoint = new
8                     ↪ IPEndPoint(IPAddress.Any, 0);
9                 byte[] data = udpClient.Receive(ref senderEndPoint);
10
11                 string json = Encoding.UTF8.GetString(data);
12                 NetworkMessage message =
13                     ↪ JsonUtility.FromJson<NetworkMessage>(json);
14
15                 if (role == NetworkRole.Server)
16                 {
17                     RegisterClientEndPoint(message, senderEndPoint);
18                 }
19
20                 lock (queueLock)
21                 {
22                     receivedMessages.Enqueue(message);
23                 }
24             }
25             catch
26             {
27             }
28         }
29     }
```

Продолжение листинга 3.2

```
26     }
27 }
28
29 private void RegisterClientEndPoint(NetworkMessage message,
   ↪ IPEndPoint senderEndPoint)
30 {
31     if (message == null)
32         return;
33
34     string playerId = "";
35
36     if (message.messageType == NetworkMessageType.ClientHello)
37     {
38         playerId = message.senderPlayerId;
39     }
40     else if (message.puzzleEvent != null)
41     {
42         playerId = message.puzzleEvent.sourcePlayerId;
43     }
44
45     if (string.IsNullOrEmpty(playerId))
46         return;
47
48     lock (clientsLock)
49     {
50         clientEndpoints[playerId] = senderEndPoint;
51     }
52 }
53
54 private void ProcessReceivedMessagesOnMainThread()
55 {
56     while (true)
57     {
58         NetworkMessage message = null;
59
60         lock (queueLock)
61         {
62             if (receivedMessages.Count > 0)
63             {
64                 message = receivedMessages.Dequeue();
```

Окончание листинга 3.2

```
65         }
66     }
67
68     if (message == null)
69         break;
70
71     Debug.Log("[UDP " + role + "] Received message type: " +
72         ↳ message.messageType);
73
74     if (EventFsmSyncManager.Instance != null)
75     {
76         EventFsmSyncManager.Instance.OnNetworkMessageReceived(
77             ↳ message);
78     }
79 }
```

3.4 Реализация структуры сетевого сообщения

Сетевое сообщение представлено классом `NetworkMessage`. В реализации выделены три типа сообщений:

- `ClientHello` — служебное сообщение регистрации клиента;
- `PuzzleEvent` — сообщение с игровым событием;
- `Acknowledgement` — подтверждение обработки события.

Тип сообщения задается перечислением `NetworkMessageType`. Для создания сообщений используются статические методы:

- `CreateClientHelloMessage`;
- `CreatePuzzleEventMessage`;
- `CreateAcknowledgementMessage`.

Игровое событие представлено классом `PuzzleEvent`. Событие содержит идентификатор, номер последовательности, идентификатор игрока-

отправителя, идентификатор исходного объекта, идентификатор целевого объекта и тип события. Тип события задается перечислением `PuzzleEventType`.

Использование единого типа `PuzzleEvent` позволяет обрабатывать игровые события одинаковым образом независимо от того, к какому объекту сцены они относятся. Конкретная логика перехода состояния реализуется внутри целевого объекта, зарегистрированного в менеджере синхронизации.

Реализация структуры сетевого сообщения и фабричных методов создания сообщений приведена в листинге 3.3.

Листинг 3.3 – Класс сетевого сообщения

```
1 [System.Serializable]
2 public class NetworkMessage
3 {
4     public NetworkMessageType messageType;
5
6     public PuzzleEvent puzzleEvent;
7
8     public int ackForEventId;
9     public string senderPlayerId;
10    public string targetPlayerId;
11
12    public NetworkMessage(NetworkMessageType messageType)
13    {
14        this.messageType = messageType;
15        this.puzzleEvent = null;
16        this.ackForEventId = -1;
17        this.senderPlayerId = "";
18        this.targetPlayerId = "";
19    }
20
21    public static NetworkMessage CreateClientHelloMessage(string
    ↪ playerId)
22    {
23        NetworkMessage message = new
    ↪ NetworkMessage(NetworkMessageType.ClientHello);
24        message.senderPlayerId = playerId;
25        return message;
26    }
27
```

Окончание листинга 3.3

```
28     public static NetworkMessage CreatePuzzleEventMessage(PuzzleEvent
        ↳ puzzleEvent)
29     {
30         NetworkMessage message = new
        ↳ NetworkMessage(NetworkMessageType.PuzzleEvent);
31         message.puzzleEvent = puzzleEvent;
32         message.senderPlayerId = puzzleEvent.sourcePlayerId;
33         return message;
34     }
35
36     public static NetworkMessage CreateAcknowledgementMessage(int
        ↳ ackForEventId, string targetPlayerId)
37     {
38         NetworkMessage message = new
        ↳ NetworkMessage(NetworkMessageType.Acknowledgement);
39         message.ackForEventId = ackForEventId;
40         message.targetPlayerId = targetPlayerId;
41         return message;
42     }
43 }
```

Структура игрового события приведена в листинге 3.4.

Листинг 3.4 – Класс игрового события

```
1 [System.Serializable]
2 public class PuzzleEvent
3 {
4     public int eventId;
5     public int sequenceNumber;
6
7     public string sourcePlayerId;
8     public string sourceObjectId;
9     public string targetObjectId;
10
11     public PuzzleEventType eventType;
12
13     public PuzzleEvent(
14         int eventId,
15         int sequenceNumber,
16         string sourcePlayerId,
```


Окончание листинга 3.4

```
17     string sourceObjectId,  
18     string targetObjectId,  
19     PuzzleEventType eventType)  
20 {  
21     this.eventId = eventId;  
22     this.sequenceNumber = sequenceNumber;  
23     this.sourcePlayerId = sourcePlayerId;  
24     this.sourceObjectId = sourceObjectId;  
25     this.targetObjectId = targetObjectId;  
26     this.eventType = eventType;  
27 }  
28 }
```

3.5 Реализация менеджера синхронизации событий

Основная логика метода реализована в классе `EventFsmSyncManager`. Данный класс выполняет следующие функции:

- регистрацию объектов, реализующих интерфейс `IFsmPuzzleObject`;
- создание локальных игровых событий;
- присвоение событию идентификатора и номера последовательности;
- отправку события через транспортный уровень;
- хранение неподтвержденных событий;
- повторную отправку события при отсутствии подтверждения;
- обработку входящих событий на сервере;
- обработку подтвержденных сервером событий на клиенте;
- фильтрацию дубликатов;
- контроль порядка событий от каждого игрока;
- применение события к целевому конечному автомату.

Для хранения объектов, состояние которых изменяется через конечные автоматы, используется словарь `registeredObjects`. Ключом является строковый идентификатор объекта, значением — объект, реализующий интерфейс `IFsmPuzzleObject`. Благодаря этому сетевое событие может быть доставлено конкретному объекту по полю `targetObjectId`.

Для защиты от повторной обработки используется множество `appliedEventKeys`. Ключ события формируется из идентификатора игрока-отправителя и идентификатора события. Если событие с таким ключом уже было применено, повторный пакет не изменяет состояние конечного автомата.

Для реализации подтверждений используется словарь `pendingEvents`. В нем хранятся события, отправленные клиентом, но еще не подтвержденные сервером. После получения сообщения `Acknowledgement` соответствующее событие удаляется из словаря.

Для контроля порядка сообщений используется словарь `expectedSequenceByPlayer`. Для каждого игрока хранится номер следующего ожидаемого события. Если сервер получает событие с меньшим номером, оно считается устаревшим. Если получен номер больше ожидаемого, событие считается пришедшим раньше недостающих событий и не применяется.

Фрагмент создания события, присвоения идентификатора, сохранения в журнале неподтвержденных событий и запуска повторной отправки приведен в листинге 3.5.

Листинг 3.5 – Создание события и повторная отправка до подтверждения

```
1  public void CreateAndSendEvent(string sourceObjectId, string
    ↪ targetObjectId, PuzzleEventType eventType)
2  {
3      if (role != NetworkRole.Client)
4      {
5          Debug.LogWarning("[EventFsmSyncManager] Игровые события
    ↪ должен создавать клиент. Текущая роль: " + role);
6          return;
7      }
8
9      PuzzleEvent puzzleEvent = new PuzzleEvent(
10         nextEventId++,
```

Продолжение листинга 3.5

```
11         nextSequenceNumber++,
12         localPlayerId,
13         sourceObjectId,
14         targetObjectId,
15         eventType
16     );
17
18     Debug.Log("[CLIENT " + localPlayerId + "] Создано событие: " +
19         puzzleEvent.eventType + " | ID=" + puzzleEvent.eventId
20         ↪ +
21         " | Seq=" + puzzleEvent.sequenceNumber);
22
23     if (ResearchMetrics.Instance != null)
24     {
25         ResearchMetrics.Instance.RegisterCreatedEvent(puzzleEvent ↪
26         ↪ .eventId);
27     }
28
29     if (UseReliableMethod)
30     {
31         pendingEvents[puzzleEvent.eventId] = puzzleEvent;
32     }
33
34     SendPuzzleEventMessage(puzzleEvent);
35
36     if (UseReliableMethod)
37     {
38         StartCoroutine(ResendUntilAcknowledged(puzzleEvent.eventId));
39     }
40 }
41
42 private IEnumerator ResendUntilAcknowledged(int eventId)
43 {
44     while (pendingEvents.ContainsKey(eventId))
45     {
46         yield return new WaitForSeconds(resendDelay);
47
48         if (!pendingEvents.ContainsKey(eventId))
49             yield break;
```

Окончание листинга 3.5

```
49         Debug.LogWarning("[CLIENT " + localPlayerId + "] ACK не  
        ↳ получен. Повторная отправка события ID=" + eventId);  
50  
51         if (ResearchMetrics.Instance != null)  
52         {  
53             ResearchMetrics.Instance.RegisterResend();  
54         }  
55  
56         SendPuzzleEventMessage(pendingEvents[eventId]);  
57     }  
58 }
```

Фрагмент обработки события на сервере приведен в листинге 3.6.

Листинг 3.6 – Проверка порядка и подтверждение события на сервере

```
1     private void HandlePuzzleEventOnServer(PuzzleEvent puzzleEvent)  
2     {  
3         string eventKey = GetEventKey(puzzleEvent);  
4  
5         Debug.Log("[SERVER] Получено событие: " + puzzleEvent.eventType +  
6             " | Key=" + eventKey +  
7             " | Seq=" + puzzleEvent.sequenceNumber +  
8             " | Player=" + puzzleEvent.sourcePlayerId);  
9  
10        if (appliedEventKeys.Contains(eventKey))  
11        {  
12            Debug.LogWarning("[SERVER] Повтор события. FSM не меняем,  
            ↳ отправляем только ACK: " + eventKey);  
13  
14            if (ResearchMetrics.Instance != null)  
15            {  
16                ResearchMetrics.Instance.RegisterDuplicateRejected();  
17            }  
18  
19            SendAcknowledgement(puzzleEvent.eventId,  
            ↳ puzzleEvent.sourcePlayerId);  
20            return;  
21        }  
22    }
```

Продолжение листинга 3.6

```
23     if (!expectedSequenceByPlayer.ContainsKey(puzzleEvent ↵
24         ↵ .sourcePlayerId))
25     {
26         expectedSequenceByPlayer[puzzleEvent.sourcePlayerId] = 1;
27     }
28
29     int expectedSequence =
30     ↵ expectedSequenceByPlayer[puzzleEvent.sourcePlayerId];
31
32     if (puzzleEvent.sequenceNumber < expectedSequence)
33     {
34         Debug.LogWarning("[SERVER] Старое событие. Ожидали Seq=" +
35             ↵ expectedSequence + ", получили Seq=" +
36             ↵ puzzleEvent.sequenceNumber);
37
38         if (ResearchMetrics.Instance != null)
39         {
40             ResearchMetrics.Instance.RegisterWrongOrderRejected();
41         }
42
43         SendAcknowledgement(puzzleEvent.eventId,
44             ↵ puzzleEvent.sourcePlayerId);
45         return;
46     }
47
48     if (puzzleEvent.sequenceNumber > expectedSequence)
49     {
50         Debug.LogWarning("[SERVER] Событие пришло слишком рано.
51             ↵ Ожидали Seq=" +
52             ↵ expectedSequence + ", получили Seq=" +
53             ↵ puzzleEvent.sequenceNumber);
54
55         if (ResearchMetrics.Instance != null)
56         {
57             ResearchMetrics.Instance.RegisterWrongOrderRejected();
58         }
59
60         return;
61     }
62 }
```

Окончание листинга 3.6

```
57         bool success = ApplyEventToFsm(puzzleEvent);
58
59         if (!success)
60         {
61             Debug.LogWarning("[SERVER] Событие не прошло проверку FSM и
        ↳ не будет подтверждено");
62             return;
63         }
64
65         appliedEventKeys.Add(eventKey);
66         expectedSequenceByPlayer[puzzleEvent.sourcePlayerId] =
        ↳ expectedSequence + 1;
67
68         if (ResearchMetrics.Instance != null)
69         {
70             ResearchMetrics.Instance.RegisterAppliedEvent();
71         }
72
73         Debug.Log("[SERVER] Событие применено. Следующий ожидаемый Seq
        ↳ для " +
74                 puzzleEvent.sourcePlayerId + " = " +
75                 expectedSequenceByPlayer[puzzleEvent.sourcePlayerId]);
76
77         SendAcknowledgement(puzzleEvent.eventId,
        ↳ puzzleEvent.sourcePlayerId);
78         SendPuzzleEventMessage(puzzleEvent);
79     }
```

3.6 Реализация подтверждений и повторной отправки

После создания события клиент сохраняет его в `pendingEvents`, отправляет сообщение `PuzzleEvent` и запускает корутину `ResendUntilAcknowledged`. Пока событие присутствует в словаре неподтвержденных событий, корутина ожидает интервал `resendDelay` и повторно отправляет то же самое событие.

Повторно отправляемое событие сохраняет исходные значения `eventId` и `sequenceNumber`. Это необходимо, потому что повторная отправка не должна восприниматься системой как новое игровое действие. Если получатель уже

применил событие, он распознает дубликат и не изменяет состояние повторно.

Сервер после успешной проверки и применения события отправляет клиенту подтверждение. Подтверждение содержит идентификатор события и идентификатор игрока, которому оно адресовано. Клиент принимает подтверждение только в том случае, если поле `targetPlayerId` совпадает с его локальным идентификатором.

Фрагмент обработки подтверждения на стороне клиента приведен в листинге 3.7.

Листинг 3.7 – Обработка подтверждения и удаление события из журнала ожидания

```
1  private void HandleAcknowledgement(NetworkMessage message)
2  {
3      if (role != NetworkRole.Client)
4          return;
5
6      if (message.targetPlayerId != localPlayerId)
7          return;
8
9      if (pendingEvents.TryGetValue(message.ackForEventId, out
10         ↪ PuzzleEvent acknowledgedEvent))
11      {
12          pendingEvents.Remove(message.ackForEventId);
13
14          Debug.Log("[CLIENT " + localPlayerId + "] Получен АСК.
15             ↪ Событие удалено из очереди ожидания: " +
16             ↪ message.ackForEventId);
17
18          if (ResearchMetrics.Instance != null)
19          {
20              ResearchMetrics.Instance.RegisterAck(message.
21                 ↪ .ackForEventId);
22          }
23
24          string eventKey = GetEventKey(acknowledgedEvent);
25
26          if (!appliedEventKeys.Contains(eventKey))
27          {
28              bool success = ApplyEventToFsm(acknowledgedEvent);
```

Окончание листинга 3.7

```
25
26         if (success)
27         {
28             appliedEventKeys.Add(eventKey);
29
30             if (ResearchMetrics.Instance != null)
31             {
32                 ResearchMetrics.Instance.RegisterAppliedEvent();
33             }
34
35             Debug.Log("[CLIENT " + localPlayerId + "] Собственное
↪ событие применено по ACK: " + eventKey);
36         }
37     else
38     {
39         Debug.LogWarning("[CLIENT " + localPlayerId + "] ACK
↪ получен, но собственное событие не применено
↪ локально: " + eventKey);
40     }
41 }
42 }
43 else
44 {
45     Debug.LogWarning("[CLIENT " + localPlayerId + "] ACK получен,
↪ но событие уже не ожидалось: " + message.ackForEventId);
46 }
47 }
```

3.7 Реализация объектов с конечными автоматами

Каждый игровой объект, состояние которого изменяется через сетевое событие, реализует интерфейс `IFsmPuzzleObject`. Интерфейс содержит свойство `ObjectId` и метод `ApplyPuzzleEvent`. Такой подход позволяет отделить общий механизм сетевой синхронизации от конкретной игровой логики.

При запуске объект регистрируется в `EventFsmSyncManager` вызовом метода `RegisterObject`. Менеджер сохраняет объект в словаре `registeredObjects`, где ключом является строковый идентификатор `ObjectId`. После получения подтвержденного события менеджер находит це-

левой объект по полю `targetObjectId` и вызывает метод `ApplyPuzzleEvent`.

В разработанном программном обеспечении классы `PuzzleDoor`, `ToggleBridge`, `CoopHoldPuzzleController` и `ColorSequenceController` выступают как примеры объектов, подключаемых к методу через единый интерфейс. Их внутренняя логика может различаться, однако для алгоритма синхронизации все такие объекты не отличаются, они предоставляют идентификатор и возвращают результат применения события. Это позволяет рассматривать конкретные игровые сценарии как заменяемые реализации целевого конечного автомата, не изменяющие транспортный уровень и механизм согласования состояний.

Интерфейс синхронизируемого объекта приведен в листинге 3.8.

Листинг 3.8 – Интерфейс объекта с конечным автоматом

```
1 public interface IFsmPuzzleObject
2 {
3     string ObjectId { get; }
4
5     bool ApplyPuzzleEvent(PuzzleEvent puzzleEvent);
6 }
```

Пример реализации объекта с конечным автоматом приведен в листинге 3.9.

Листинг 3.9 – Регистрация и обработка события объектом `PuzzleDoor`

```
1 public class PuzzleDoor : MonoBehaviour, IFsmPuzzleObject
2 {
3     [Header("FSM Object Settings")]
4     public string objectId = "Door_A";
5
6     [Header("Door Movement")]
7     public float openHeight = 3f;
8     public float moveSpeed = 3f;
9
10    private DoorStateMachine stateMachine;
11    private Vector3 closedPosition;
12    private Vector3 openPosition;
13    private Coroutine moveCoroutine;
14 }
```

Продолжение листинга 3.9

```
15     private bool isInitialized;
16
17     public string ObjectId => objectId;
18
19     private void Awake()
20     {
21         InitializeDoor();
22     }
23
24     private void Start()
25     {
26         if (EventFsmSyncManager.Instance != null)
27         {
28             EventFsmSyncManager.Instance.RegisterObject(this);
29         }
30         else
31         {
32             Debug.LogError("[PuzzleDoor] EventFsmSyncManager не найден на
                 ↳ сцене");
33         }
34     }
35
36     private void InitializeDoor()
37     {
38         if (isInitialized)
39             return;
40
41         closedPosition = transform.position;
42         openPosition = closedPosition + Vector3.up * openHeight;
43
44         stateMachine = new DoorStateMachine();
45
46         isInitialized = true;
47
48         Debug.Log("[PuzzleDoor] Инициализирована дверь: " + objectId);
49     }
50
51     public bool ApplyPuzzleEvent(PuzzleEvent puzzleEvent)
52     {
53         InitializeDoor();
```

```

54
55     if (puzzleEvent.eventType == PuzzleEventType.ButtonPressed)
56     {
57         return OpenByFsm();
58     }
59
60     Debug.LogWarning("[PuzzleDoor] Неподходящее событие для двери: "
61         ↳ + puzzleEvent.eventType);
62     return false;
63 }

```

3.8 Реализация пользовательского взаимодействия

Пользовательское взаимодействие реализовано через интерактивные объекты сцены. Объекты, с которыми может взаимодействовать игрок, преобразуют локальное действие в вызов `CreateAndSendEvent`. При этом локальное действие не передается по сети напрямую, оно сначала приводится к типизированному событию `PuzzleEvent`, содержащему идентификаторы источника, цели, игрока и тип операции.

Таким образом, пользовательское действие не изменяет состояние удаленного объекта напрямую. Оно преобразуется в событие, которое проходит через общий механизм сетевой синхронизации и только после проверки применяется к целевому конечному автомату.

3.9 Средства имитации сетевых ошибок

Для проверки устойчивости метода в транспортном модуле реализованы параметры искусственного ухудшения сети:

- `outgoingPacketLossChance` — вероятность потери исходящего пакета;
- `artificialLatencyMs` — средняя искусственная задержка отправки;
- `jitterMs` — разброс задержки;
- `duplicatePacketChance` — вероятность отправки дубликата.

Эти параметры задаются в инспекторе Unity и позволяют моделировать нестабильное сетевое соединение без изменения основного алгоритма. При

имитации потери сообщение не отправляется в сеть. При имитации задержки отправка откладывается на заданное время. При имитации дубликата тот же массив байтов отправляется повторно через небольшой интервал.

Сбор статистики выполняется классом **ResearchMetrics**. Он регистрирует количество отправленных сообщений, имитированных потерь, повторных отправок, отклоненных дубликатов, событий с нарушением порядка и успешно примененных событий. Также сохраняется время создания события и время получения подтверждения, что позволяет вычислять среднее время подтверждения.

Фрагмент регистрации метрик и вычисления среднего времени подтверждения приведен в листинге 3.10.

Листинг 3.10 – Сбор исследовательских метрик

```
1  public void RegisterSendMessage()  
2  {  
3      sentMessages++;  
4  }  
5  
6  public void RegisterPacketLoss()  
7  {  
8      lostPackets++;  
9  }  
10  
11 public void RegisterResend()  
12 {  
13     resendCount++;  
14 }  
15  
16 public void RegisterDuplicateRejected()  
17 {  
18     duplicateRejected++;  
19 }  
20  
21 public void RegisterWrongOrderRejected()  
22 {  
23     wrongOrderRejected++;  
24 }  
25  
26 public void RegisterAppliedEvent()
```

Окончание листинга 3.10

```
27     {
28         appliedEvents++;
29     }
30
31     public void RegisterCreatedEvent(int eventId)
32     {
33         createdEvents++;
34         eventCreateTimes[eventId] = Time.time;
35     }
36
37     public void RegisterAck(int eventId)
38     {
39         if (!eventCreateTimes.ContainsKey(eventId))
40             return;
41
42         float delay = Time.time - eventCreateTimes[eventId];
43
44         ackTimes.Add(delay);
45
46         Debug.Log("[RESEARCH] RTT for Event " +
47                 eventId +
48                 " = " +
49                 delay.ToString("F3") +
50                 " sec");
51     }
52
53     public float GetAverageAckTime()
54     {
55         if (ackTimes.Count == 0)
56             return 0f;
57
58         float sum = 0f;
59
60         foreach (float time in ackTimes)
61         {
62             sum += time;
63         }
64
65         return sum / ackTimes.Count;
66     }
```

3.10 Тестирование разработанного программного обеспечения

Тестирование разработанного программного обеспечения выполнялось с использованием ручных сценариев, проверяющих основные функции метода синхронизации и корректность обработки сетевых ошибок. Для каждого сценария проверялось состояние целевого конечного автомата и сообщения журнала выполнения Unity.

Основные тестовые сценарии приведены в таблице 3.2.

Таблица 3.2 – Сценарии тестирования программной реализации

Сценарий	Действие	Ожидаемый результат
Доставка события	Игрок взаимодействует с интерактивным объектом	Создается событие, сервер применяет его к конечному автомату, клиент получает подтверждение
Повторная отправка	Для исходящих пакетов задается вероятность потери	При отсутствии подтверждения событие повторно отправляется с тем же идентификатором
Фильтрация дубликата	Включается вероятность дублирования пакета	Дубликат события не изменяет состояние повторно
Контроль порядка	Отправляется событие с номером больше ожидаемого	Событие не применяется до восстановления порядка
Недопустимое событие	Событие направляется объекту, для которого его тип не подходит	Метод ApplyPuzzleEvent возвращает отрицательный результат, состояние объекта не изменяется
Синхронизация FSM-объекта	Оба игрока формируют события для одного синхронизируемого объекта	На обеих сторонах фиксируется одинаковое логическое состояние объекта

Сценарии тестирования проверяют не отдельные игровые действия, а свойства метода: доставку критичного события, повторную отправку, фильтрацию дубликатов, контроль порядка и сохранение согласованного состояния конечного автомата.

Вывод

В данном разделе был обоснован выбор Unity, языка C# и UDP-сетевого обмена для программной реализации метода обеспечения согласованности состояний двух асинхронных игроков. Была описана структура программного обеспечения, включающая транспортный уровень, менеджер синхронизации событий, представление сетевых сообщений, интерфейс объектов с конечными автоматами и средства сбора статистики.

Были рассмотрены детали реализации прикладной надежности поверх UDP, такие как подтверждение обработки события, хранение неподтвержденных событий, повторная отправка, фильтрация дубликатов и контроль порядка сообщений. Также был описан способ подключения объектов, реализующих интерфейс `IFsmPuzzleObject`, и сценарии тестирования разработанного программного обеспечения.

4 Исследовательский раздел

4.1 Цель исследования

Целью исследования является оценка эффективности разработанного метода синхронизации состояний двух игроков и его сравнение с базовой событийной синхронизацией без прикладной надежности.

В рамках исследования необходимо:

- определить критерии оценки эффективности синхронизации состояний;
- задать параметры сетевой среды для экспериментальных запусков;
- выполнить серию испытаний для разработанного метода и базового UDP-подхода;
- оценить долю успешно примененных событий;
- оценить среднее время подтверждения или возврата события;
- оценить количество повторных отправок, возникающих при потере сообщений;
- определить влияние дублирования пакетов на повторную обработку событий;
- сравнить устойчивость двух подходов к потере, задержке и дублированию UDP-пакетов.

4.2 Сравнимые режимы синхронизации

Для проведения исследования в разработанном программном обеспечении реализованы два режима синхронизации, выбираемые параметром `SynchronizationResearchMode`. Режим `ReliableFsmMethod` соответствует разработанному методу. Режим `BasicUdpEvents` используется как контрольный вариант и моделирует простую событийную передачу по UDP без подтверждений и повторной отправки.

Сравнение выполняется при одинаковых входных событиях и одинаковых параметрах сетевой среды. Это позволяет исключить влияние различий

в игровой логике и сравнивать именно сетевые и алгоритмические механизмы синхронизации.

Таблица 4.1 – Сравнимые режимы синхронизации

Механизм	Базовая событийная UDP-синхронизация	Разработанный метод
Режим синхронизации	<code>BasicUdpEvents</code>	<code>ReliableFsmMethod</code>
Передача события	Однократная отправка события	Отправка события с регистрацией ожидания подтверждения
Подтверждение обработки	Не используется	Используется сообщение <code>Acknowledgement</code>
Повторная отправка	Не выполняется	Выполняется после истечения <code>resendDelay</code>
Фильтрация дубликатов	Не выполняется на уровне метода	Выполняется по ключу события
Контроль порядка	Не выполняется	Выполняется по <code>sequenceNumber</code>
Защита состояния FSM	Не выполняется	Обеспечивается проверкой порядка, подтверждением и повторной отправкой

В базовом режиме событие, потерянное на транспортном уровне, не восстанавливается. Если сообщение было продублировано, оно может быть повторно передано в обработчик конечного автомата. В разработанном методе потеря события или подтверждения приводит к повторной отправке, а повторное получение уже обработанного события не изменяет состояние конечного автомата.

4.3 Критерии оценки эффективности

Для оценки используются метрики, собираемые при выполнении разработанного программного обеспечения классом `ResearchMetrics`. Метрики фиксируются отдельно для каждого запуска и выводятся в журнал приложения после завершения автоматического теста.

К основным метрикам относятся:

- $N_{created}$ — количество созданных логически значимых событий;
- N_{sent} — количество фактически отправленных сетевых сообщений;
- N_{lost} — количество имитированных потерь исходящих UDP-пакетов;
- N_{resend} — количество повторных отправок событий;
- N_{dup} — количество отклоненных дубликатов;
- N_{wrong} — количество событий, отклоненных из-за нарушения порядка;
- $N_{applied}$ — количество событий, примененных к конечному автомату;
- T_{ack}^{avg} — среднее время подтверждения или возврата события.

Для разработанного метода величина T_{ack}^{avg} соответствует среднему времени получения подтверждения **Acknowledgement**. Для базового режима эта величина интерпретируется как среднее время возврата события от серверной стороны к клиенту, поскольку отдельное сообщение подтверждения в данном режиме не используется.

Среднее время подтверждения или возврата события определяется выражением:

$$T_{ack}^{avg} = \frac{1}{N} \sum_{i=1}^N (t_i^{ack} - t_i^{send}),$$

где:

- N — количество событий, для которых был получен ответ;
- t_i^{send} — момент создания события;
- t_i^{ack} — момент получения подтверждения или подтвержденного сервером события.

Доля примененных событий определяется как:

$$K_{applied} = \frac{N_{applied}}{N_{created}}.$$

Для разработанного метода ожидаемым значением $K_{applied}$ является значение, близкое к единице. Для базового режима при потере пакетов значение

$K_{applied}$ может уменьшаться, так как потерянные события не восстанавливаются. При дублировании пакетов в базовом режиме показатель может превышать единицу, что указывает на повторную обработку одного и того же события.

Относительное количество повторных отправок определяется как:

$$K_{resend} = \frac{N_{resend}}{N_{created}}.$$

Рост K_{resend} при наличии потерь является ожидаемым для разработанного метода, так как дополнительные сообщения используются для восстановления доставки критичных событий. Для базового UDP-подхода K_{resend} равно нулю, однако отсутствие повторов в этом случае не является преимуществом, так как потерянные события не компенсируются.

4.4 Порядок проведения исследования

Автоматизация испытаний выполняется компонентом `AutomatedResearchTest`. Компонент создает 100 событий с интервалом 0.2 с. После завершения генерации событий выполняется ожидание завершения сетевой обработки, затем компонент `ResearchMetrics` выводит итоговые значения метрик.

Параметры сетевой среды задаются в компоненте `UdpEventTransport`:

- `outgoingPacketLossChance` — вероятность потери исходящего пакета;
- `artificialLatencyMs` — искусственная задержка отправки;
- `jitterMs` — вариативность задержки;
- `duplicatePacketChance` — вероятность дублирования отправляемого пакета.

Для каждого сценария выполняются два запуска: один в режиме `ReliableFsmMethod`, второй в режиме `BasicUdpEvents`. Значения параметров сетевой среды в двух запусках совпадают.

4.5 Сценарии исследования

Для исследования выбраны сценарии, отражающие типовые условия сетевого взаимодействия в многопользовательском приложении. Сценарии

приведены в таблице 4.2.

Таблица 4.2 – Сценарии исследования UDP-обмена

Сценарий	Потери, %	Задержка, мс	Jitter, мс	Дубликаты, %
Стабильная сеть	0	0	0	0
Умеренная за- держка	0	100	20	0
Потери пакетов	5	100	20	0
Высокие потери	15	100	30	0
Дублирование па- кетов	0	100	20	20
Нестабильная сеть	10	150	50	10

Первые два сценария используются для оценки влияния задержки без потери сообщений. Сценарии с потерями позволяют оценить способность метода восстанавливать доставку критичных событий. Сценарий с дублированием проверяет устойчивость к повторному получению одного и того же события. Нестабильная сеть объединяет задержку, вариативность задержки, потерю и дублирование пакетов.

4.6 Результаты исследования разработанного метода

В таблице 4.3 приведены результаты запусков для режима `ReliableFsmMethod`.

Таблица 4.3 – Результаты исследования разработанного метода

Сценарий	Создано	Отправлено	Потери	Повторы	Применено	T_{ack}^{avg} , с
Стабильная сеть	100	102	0	2	100	0.099
Умеренная задержка	100	118	0	18	100	0.332
Потери пакетов	100	146	8	54	100	0.939
Высокие потери	100	132	20	46	99	0.711
Дублирование пакетов	100	131	0	12	100	0.362
Нестабильная сеть	100	139	8	38	99	0.548

Из таблицы видно, что при стабильной сети метод обеспечивает применение всех созданных событий. При увеличении задержки возрастает среднее время подтверждения, однако доля примененных событий остается равной единице. При появлении потерь возрастает количество повторных отправок, что является следствием работы механизма восстановления доставки. В сценариях с высокими потерями и нестабильной сетью часть событий может не попасть в окно наблюдения, однако механизм повторной отправки продолжает работу до получения подтверждения.

Важным результатом является то, что увеличение числа отправленных сообщений не приводит к повторному изменению состояния конечного автомата. Повторные отправки и дубликаты распознаются по ключу события и используются только для восстановления факта доставки и подтверждения обработки.

4.7 Результаты исследования базового UDP-подхода

В таблице 4.4 приведены результаты запусков для режима `BasicUdpEvents`. В этом режиме событие отправляется один раз, не помещается в журнал неподтвержденных событий и не отправляется повторно.

Таблица 4.4 – Результаты исследования базовой событийной UDP-синхронизации

Сценарий	Создано	Отправлено	Потери	Повторы	Применено	T_{ack}^{avg} , с
Стабильная сеть	100	100	0	0	100	0.086
Умеренная за- держка	100	100	0	0	100	0.301
Потери пакетов	100	95	5	0	91	0.314
Высокие потери	100	85	15	0	73	0.327
Дублирование пакетов	100	120	0	0	124	0.309
Нестабильная сеть	100	101	10	0	88	0.423

При отсутствии потерь базовый подход способен доставить все события, так как сетевой канал не создает отказов. Однако при потере пакетов доля примененных событий снижается, поскольку потерянное событие не восстанавливается. При дублировании пакетов количество применений может превышать число созданных событий, что свидетельствует о повторной обработке одного и того же логического действия.

4.8 Сравнение результатов

Сводное сравнение двух режимов приведено в таблице 4.5. Для сравнения используется доля примененных событий $K_{applied}$. Для разработанного метода значение, близкое к единице, означает доставку и обработку созданных событий. Для базового подхода значение меньше единицы означает потерю событий, а значение больше единицы — повторную обработку из-за дублирования.

Таблица 4.5 – Сравнение разработанного метода с базовой UDP-синхронизацией

Сценарий	$K_{applied}$ базового подхода	$K_{applied}$ разработанного метода
Стабильная сеть	1.00	1.00
Умеренная задержка	1.00	1.00
Потери пакетов	0.91	1.00
Высокие потери	0.73	0.99
Дублирование пакетов	1.24	1.00
Нестабильная сеть	0.88	0.99

Результаты показывают, что при благоприятных сетевых условиях оба подхода дают сопоставимый результат. Это ожидаемо, поскольку при отсутствии потерь и дубликатов простая передача события достаточна для сохранения состояния. Отличие проявляется при появлении сетевых ошибок.

При потере пакетов базовый подход теряет часть событий без возможности восстановления. Разработанный метод компенсирует такие потери повторной отправкой. При дублировании пакетов базовый подход допускает повторную обработку события, тогда как разработанный метод распознает уже обработанное событие и не изменяет конечный автомат повторно.

4.9 Выводы по результатам исследования

По результатам исследования можно сделать следующие выводы:

- при стабильной сети разработанный метод и базовая событийная UDP-синхронизация обеспечивают применение всех событий;
- при увеличении задержки возрастает среднее время подтверждения, однако согласованность состояний сохраняется;
- при потере пакетов базовый подход теряет часть событий, так как не содержит механизма восстановления доставки;
- разработанный метод компенсирует потерю сообщений за счет хранения событий в `pendingEvents` и повторной отправки после истечения времени ожидания;

- при дублировании пакетов базовый подход может привести к повторной обработке одного и того же события;
- фильтрация дубликатов и контроль порядка в разработанном методе предотвращают повторное и несогласованное изменение конечного автомата;
- увеличение числа отправленных сообщений в разработанном методе является ожидаемым следствием обеспечения прикладной надежности поверх UDP.

Вывод

В данном разделе были определены критерии оценки эффективности разработанного метода, описаны режимы экспериментального сравнения и заданы сценарии исследования UDP-сетевого обмена. Было проведено сравнение разработанного метода с базовой событийной UDP-синхронизацией без прикладной надежности.

Результаты исследования показывают, что базовый подход достаточен только при отсутствии сетевых ошибок. При потере и дублировании пакетов он не гарантирует ни доставку критичного события, ни его однократную обработку. Разработанный метод вводит дополнительные служебные механизмы: подтверждения, повторную отправку, контроль порядка и фильтрацию дубликатов. Эти механизмы увеличивают количество сетевых сообщений, однако позволяют сохранять согласованность состояний конечного автомата в условиях ненадежного UDP-обмена.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы был разработан метод обеспечения согласованности состояний двух асинхронных игроков с помощью конечных автоматов.

В ходе выполнения работы были решены следующие задачи:

- проведен анализ предметной области синхронизации игровых состояний и передачи данных в многопользовательских играх;
- рассмотрены существующие подходы к синхронизации состояний и способы передачи игровых данных;
- описаны типовые игровые данные, подлежащие хранению и передаче;
- определены основные причины рассинхронизации игровых состояний;
- разработан метод синхронизации состояний двух игроков с использованием конечных автоматов;
- определена формальная модель состояний системы, событий и переходов между состояниями;
- разработана структура конечного автомата, описывающего взаимодействие игроков;
- представлен алгоритм синхронизации состояний, включающий обработку входящих и исходящих событий, подтверждения доставки и повторную отправку сообщений;
- обоснован выбор Unity, языка C# и UDP-сетевого обмена в качестве средств программной реализации;
- разработано программное обеспечение, реализующее предложенный метод синхронизации;
- разработано программное обеспечение для взаимодействия игроков на основе событий и конечных автоматов;
- проведено тестирование разработанного программного обеспечения;

- определены критерии оценки эффективности разработанного метода;
- разработана методика исследования эффективности метода при различных параметрах сетевого взаимодействия;
- выполнено сравнение разработанного метода с событийной синхронизацией без механизмов прикладной надежности.

Разработанный метод рассматривает синхронизацию не как простую передачу состояния между клиентом и сервером, а как задачу согласованного выполнения событий в распределенной асинхронной системе. Для этого логика взаимодействия игроков представлена в виде конечного автомата, а каждое значимое игровое действие обрабатывается как событие, допустимость которого проверяется относительно текущего состояния системы.

В программной реализации поверх UDP-сетевого обмена были реализованы механизмы прикладной надежности, такие как подтверждение обработки событий, хранение неподтвержденных событий, повторная отправка при отсутствии подтверждения, фильтрация дубликатов и контроль порядка сообщений. Эти механизмы позволяют снизить влияние потерь, дублирования и нарушения порядка доставки пакетов на корректность логического состояния игроков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Smed J., Hakonen H.* Aspects of Networking in Multiplayer Computer Games. — The electronic library, 2002.
2. *Tanenbaum A. S., Van Steen M.* Distributed Systems. — Pearson, 2017.
3. *Gambetta G.* Fast-Paced Multiplayer (Part I): Client-Server Game Architecture [Электронный ресурс]. — URL: [https : / / www . gabrielgambetta.com/client-server-game-architecture.html](https://www.gabrielgambetta.com/client-server-game-architecture.html) (дата обращения: 23.11.2025).
4. *Corporation V.* Source Multiplayer Networking [Электронный ресурс]. — 2021. — URL: https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking (дата обращения: 20.11.2025).
5. *Technologies U.* Unity Netcode for GameObjects Documentation [Электронный ресурс]. — 2025. — URL: <https://docs-multiplayer.unity3d.com> (дата обращения: 22.11.2025).
6. *Games E.* Unreal Engine Networking and Replication Guide [Электронный ресурс]. — 2025. — URL: <https://dev.epicgames.com/> (дата обращения: 19.11.2025).
7. *Furht B.* Encyclopedia of Multimedia. — Springer, 2008.
8. *Engine P.* Photon Quantum Deterministic Simulation Documentation [Электронный ресурс]. — 2024. — URL: <https://doc.photonengine.com> (дата обращения: 17.11.2025).
9. *Claypool M., Claypool K.* Latency Can Kill: Precision and Deadline in Online Games // ACM Computers in Entertainment. — 2006.
10. *Cannon T.* Fight the Lag! The Trick Behind GGPO's Low Latency Netcode // Game Developer. — 2012.
11. *Kurose J. F., Ross K. W.* Computer Networking: A Top-Down Approach. — Pearson, 2013.
12. *Stevens W. R., Fall K. R.* TCP/IP Illustrated. — Addison-Wesley, 2011.
13. *Gregory J.* Game Engine Architecture. — 3rd ed. — A K Peters / CRC Press, 2018.

14. *Pantel A., Wolf L. C.* On the Impact of Delay on Real-Time Multiplayer Games // Proceedings of NOSSDAV. — 2002.
15. *Gambetta G.* Fast-Paced Multiplayer (Part IV): Lag Compensation [Электронный ресурс]. — URL: <https://www.gabrielgambetta.com/lag-compensation.html> (дата обращения: 17.11.2025).
16. MonoBehaviour [Электронный ресурс]. — Режим доступа URL: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html> (Дата обращения: 13.05.2026).
17. Документация по C# [Электронный ресурс]. — Режим доступа URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (Дата обращения: 13.05.2026).
18. UdpClient Class [Электронный ресурс]. — Режим доступа URL: <https://learn.microsoft.com/ru-ru/dotnet/api/system.net.sockets.udpclient> (Дата обращения: 13.05.2026).

ПРИЛОЖЕНИЕ А